

2-4 Trees, Red Black Trees, AVL Trees

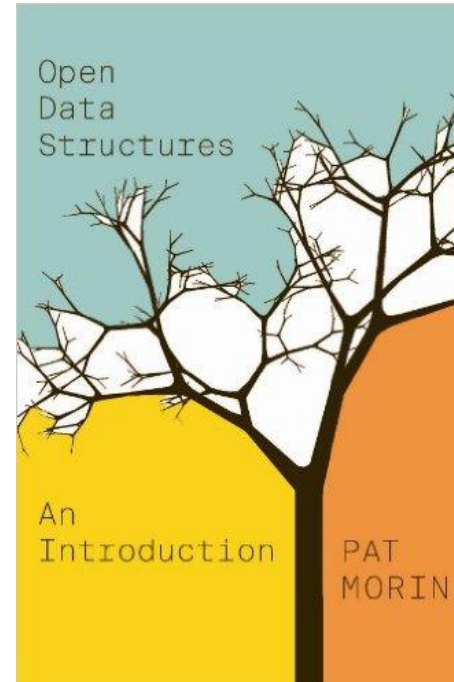
COMP 2402

Abstract Data Types & Algorithms

Readings

Today's topic

- 2-4 Trees
- Red Black Trees



Sorted Sets

find, add, remove

Skip Lists

expected $O(\log n)$

Binary Search Trees

$O(\text{height}) \in O(n)$

Random BST

expected $O(\log n)$ [find]

Treap

expected $O(\log n)$

Scapegoat Trees

$O(\log n)$ [find, add*, remove*]

amortized $O(\log n)$ [rebuild]

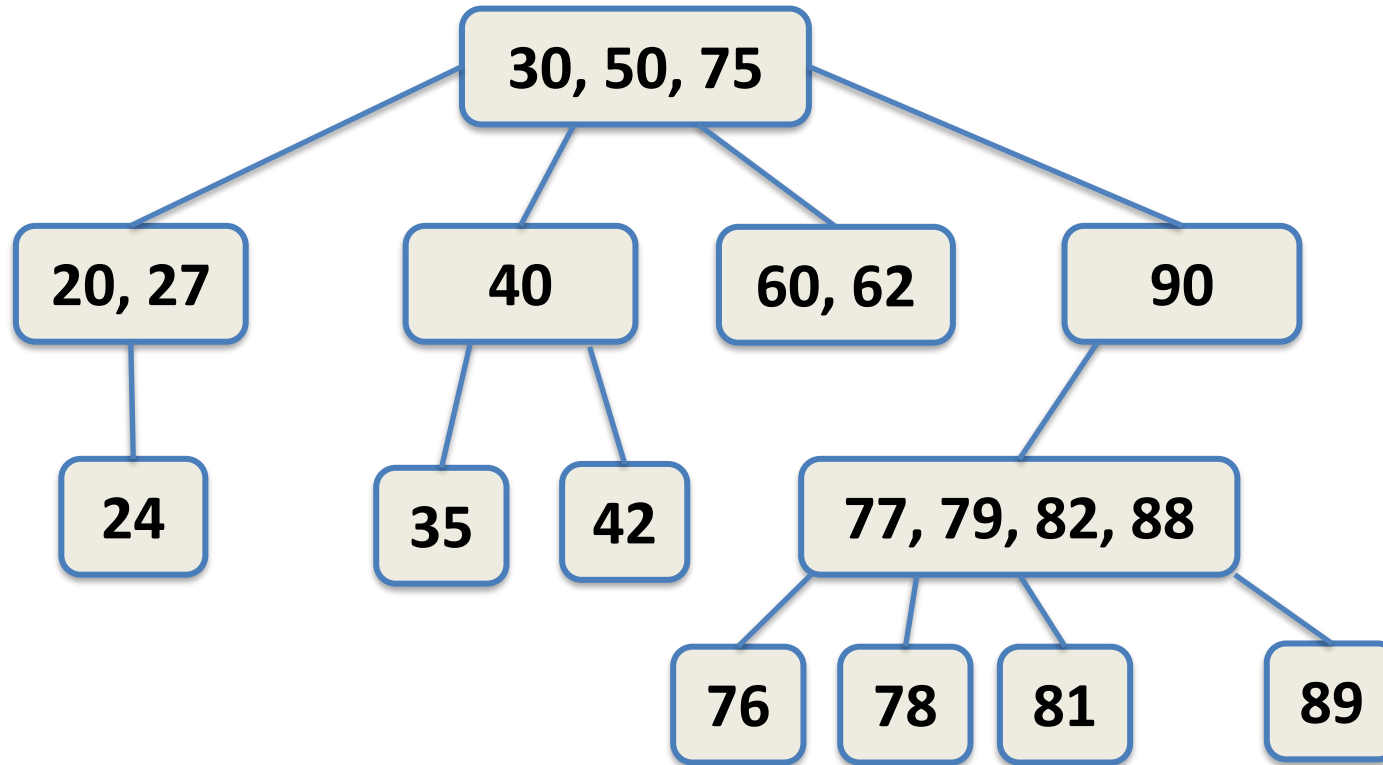
Multi-Way Search Trees

Recall a Binary Search Tree BST:

- Each node stores one value (or key)
- Each node has 0, 1 or 2 children
 - value of a node is greater than the value of every node in its left subtree
 - Value of a node is less than the value of every node in its right subtree

Multi-Way Search Trees

A **multi-way search tree** is a generalization of a binary search tree where nodes can hold more than one value (key) and have more than two children.



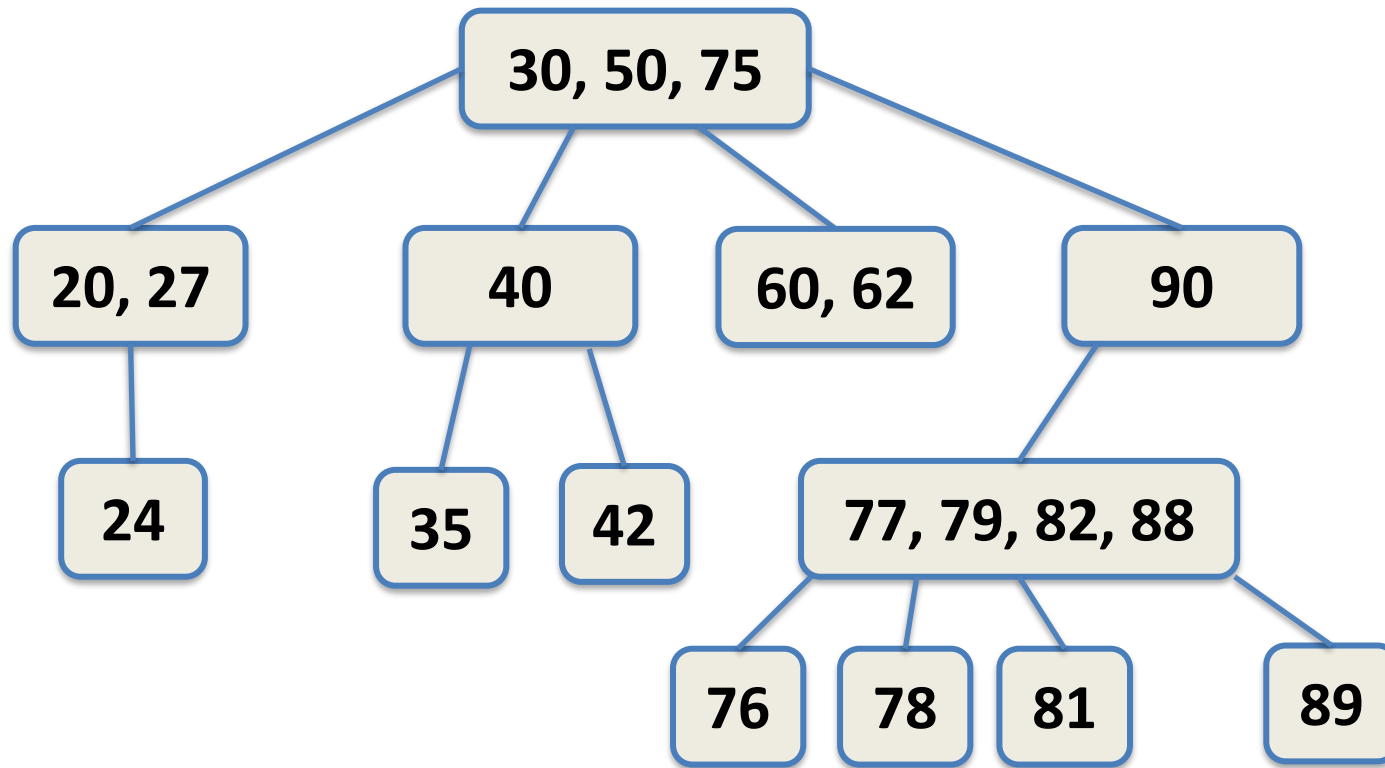
Multi-Way Search Trees

- If a node has d values $v_1, v_2, \dots, v_d$ then it can have up to $d + 1$ children $c_1, c_2, \dots, c_{d+1}$
- Let $v_0 = -\infty, v_{d+1} = \infty$. The value of all nodes in the subtree rooted at c_i must satisfy

$$v_{i-1} < \text{value}(c_i) < v_i$$

- External nodes are just placeholders.

Multi-Way Search Trees



2-4 Trees

A **2-4 Tree** is a multi-way search tree in which

- All internal nodes have 1,2 or 3 values
($\therefore$ all internal nodes have 2, 3 or 4 children)
- All external nodes (leaves) in a 2-4 tree must have the same depth.

2-4 Trees

Exercise: Prove that a 2-4 tree with n leaves has height at most $\log_2 n$.

Question: How do we maintain the properties when adding/removing elements from the tree?

2-4 Trees

Adding a value (key) to a 2-4 tree.

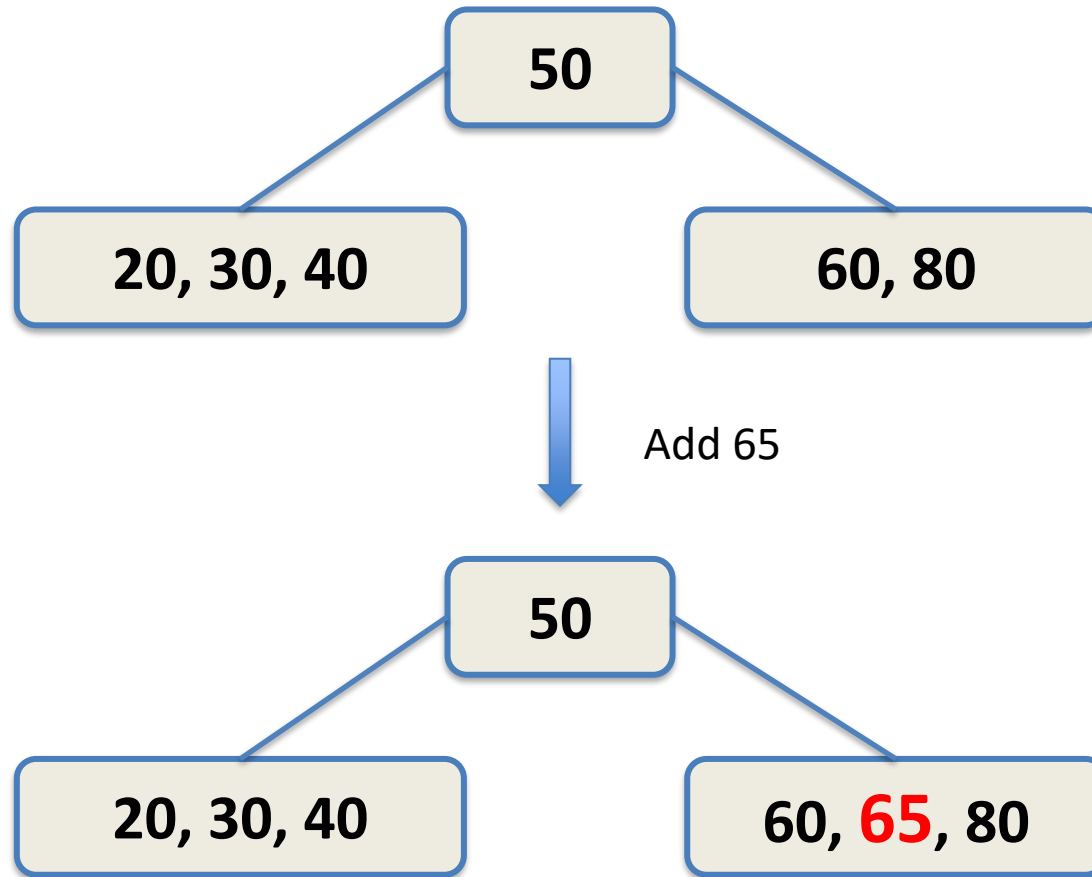
Just as with a BST you search for the element you want to add and then add it at the bottom of the tree where it would have been if it was already there.

If the node has 1 or 2 values in it then this is easy.

If the node already has 3 values in it this is an **overflow**.

2-4 Trees

If the node has 1 or 2 values in it then this is easy.



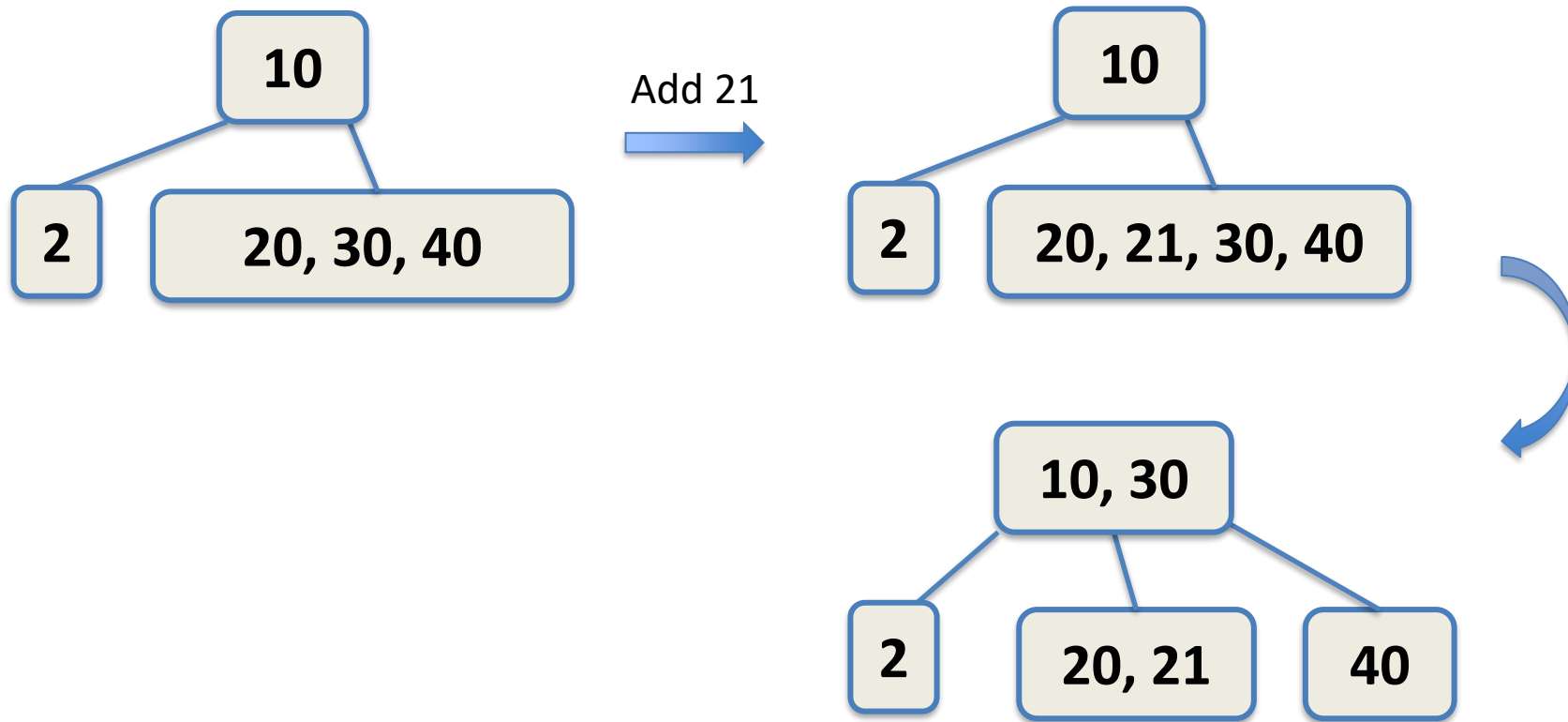
2-4 Trees

If an **overflow** occurs when trying to add a value to a node u then we **split** the node into two nodes u_1, u_2 such that

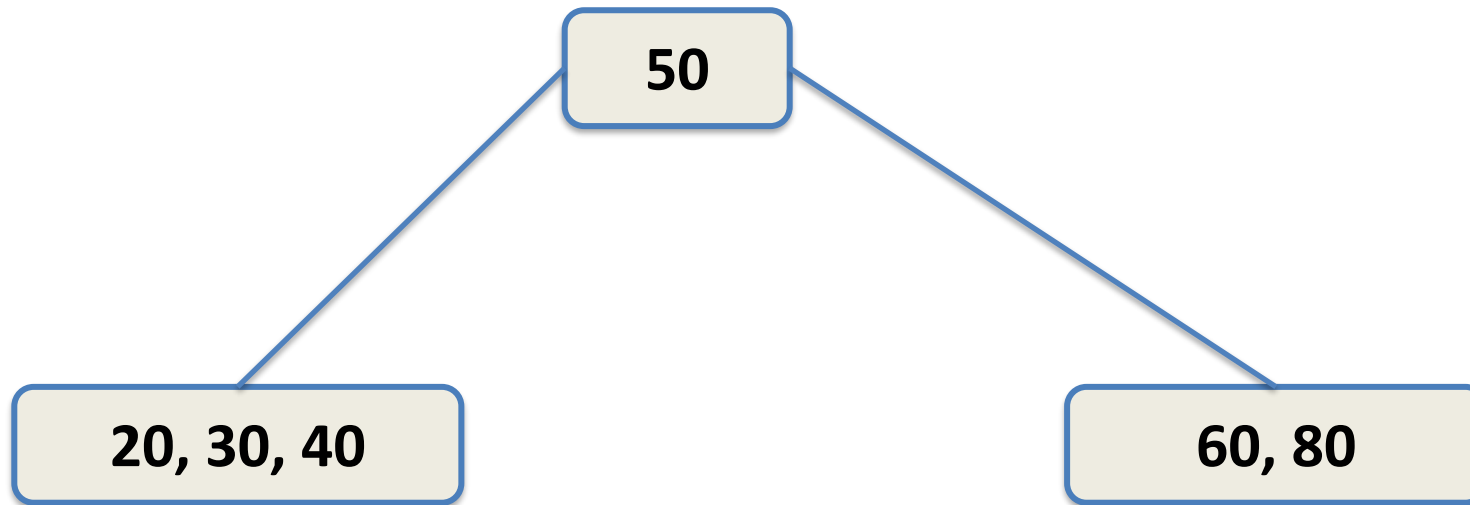
- u_1 contains the 2 smallest values (of the 4)
- u_2 contains the largest values (of the 4)
- The 3rd value is pushed up the parent node (which might cause that node to overflow)

2-4 Trees

overflow → eject 3rd value to parent and **split**

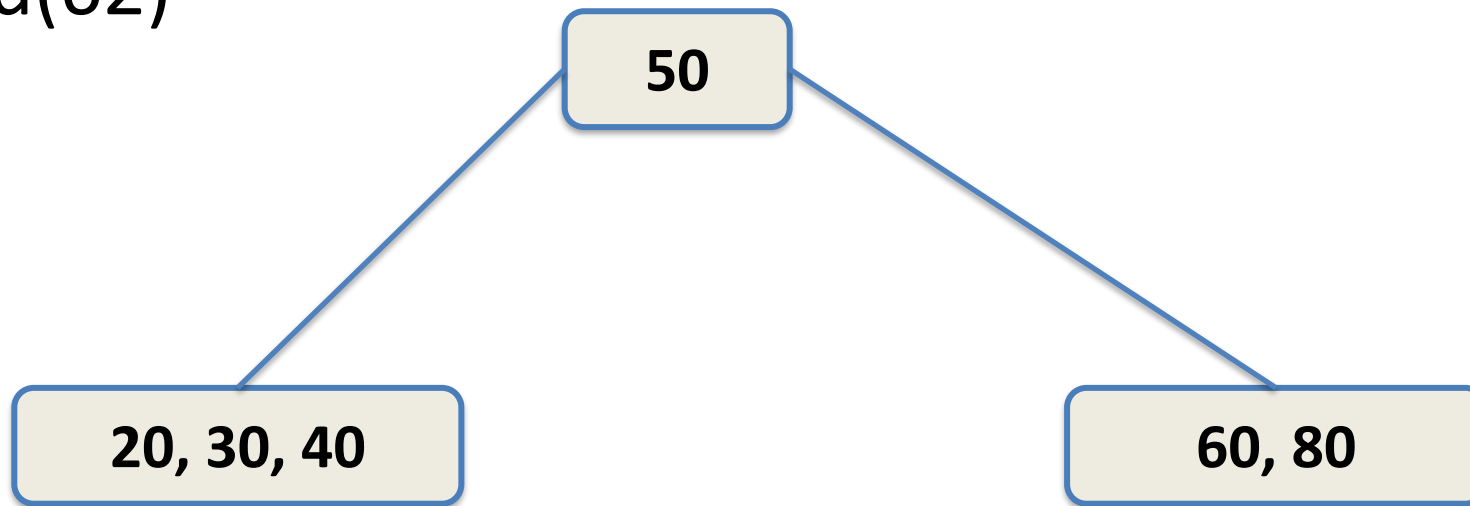


Adding to a 2-4 Trees



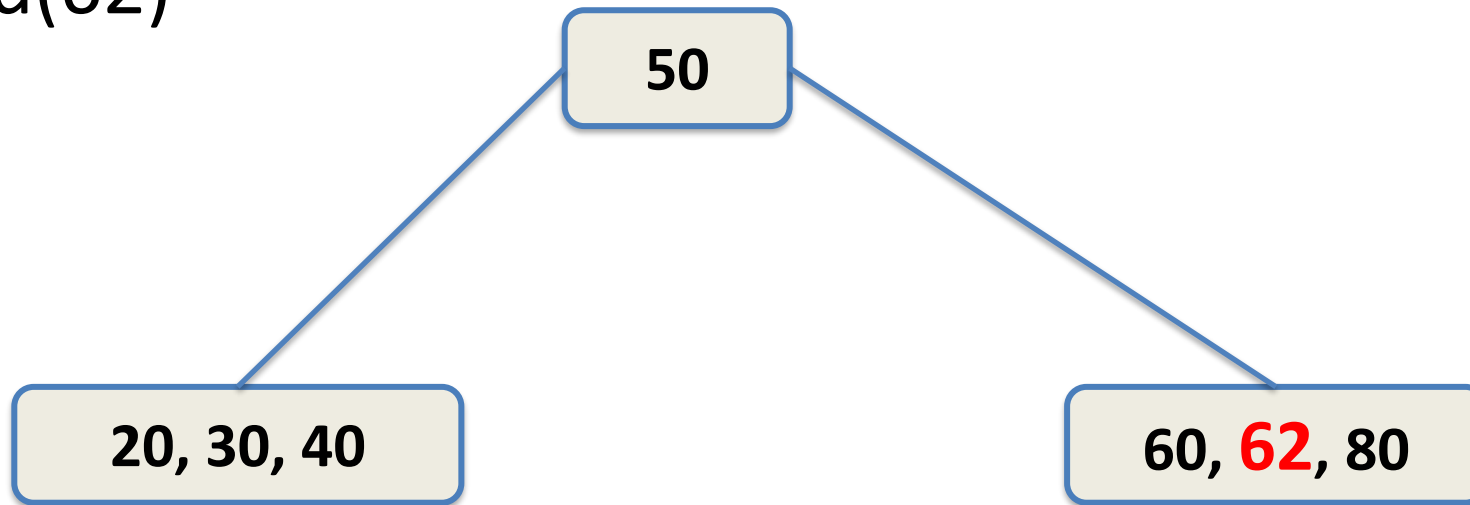
Adding to a 2-4 Trees

Add(62)



Adding to a 2-4 Trees

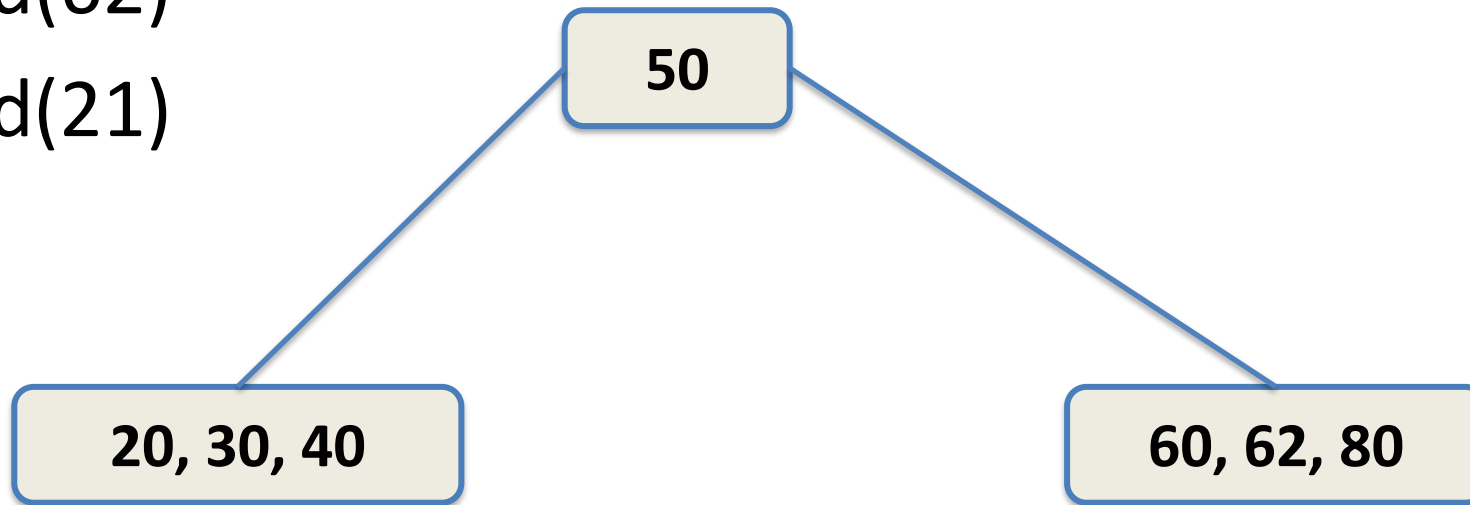
Add(62)



Adding to a 2-4 Trees

Add(62)

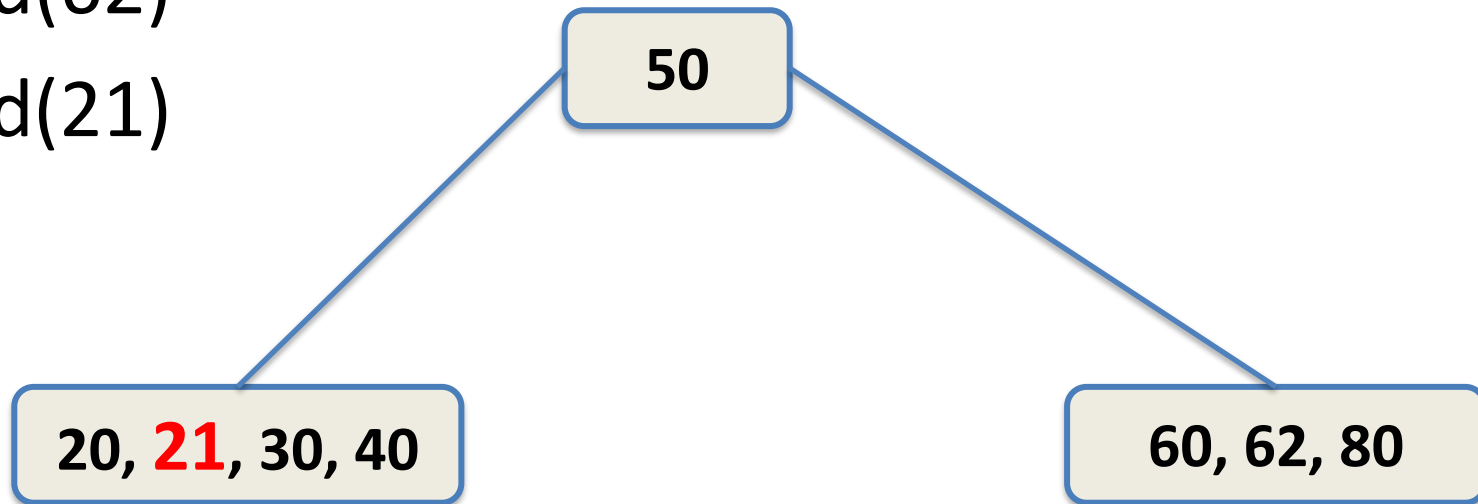
Add(21)



Adding to a 2-4 Trees

Add(62)

Add(21)



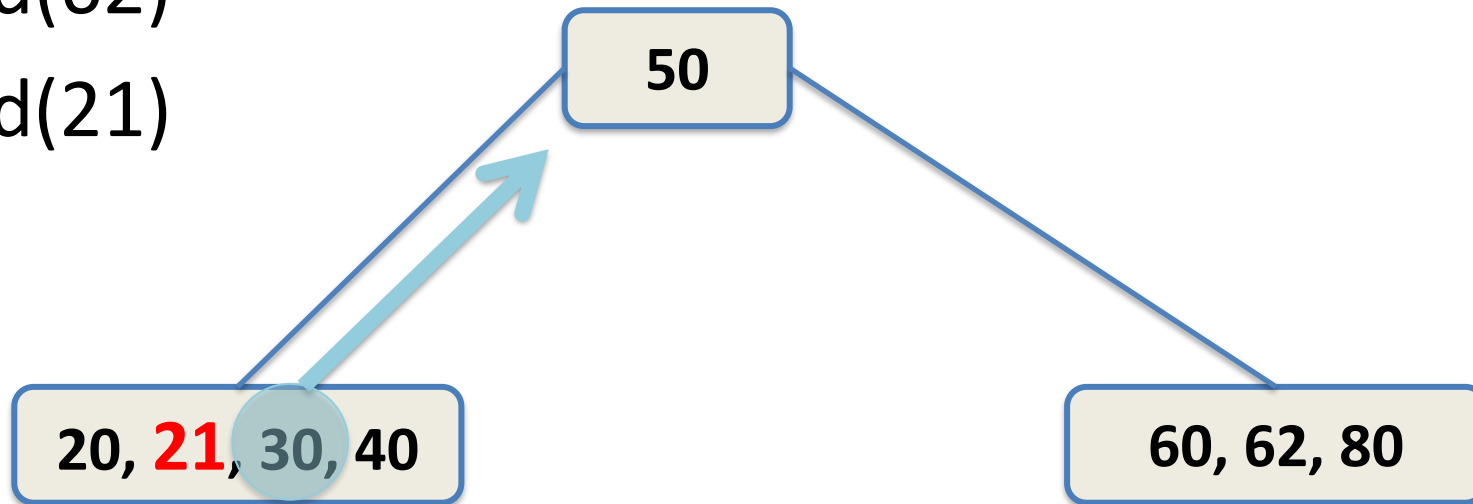
Overflow!

Eject 30 and **split** what's left

Adding to a 2-4 Trees

Add(62)

Add(21)



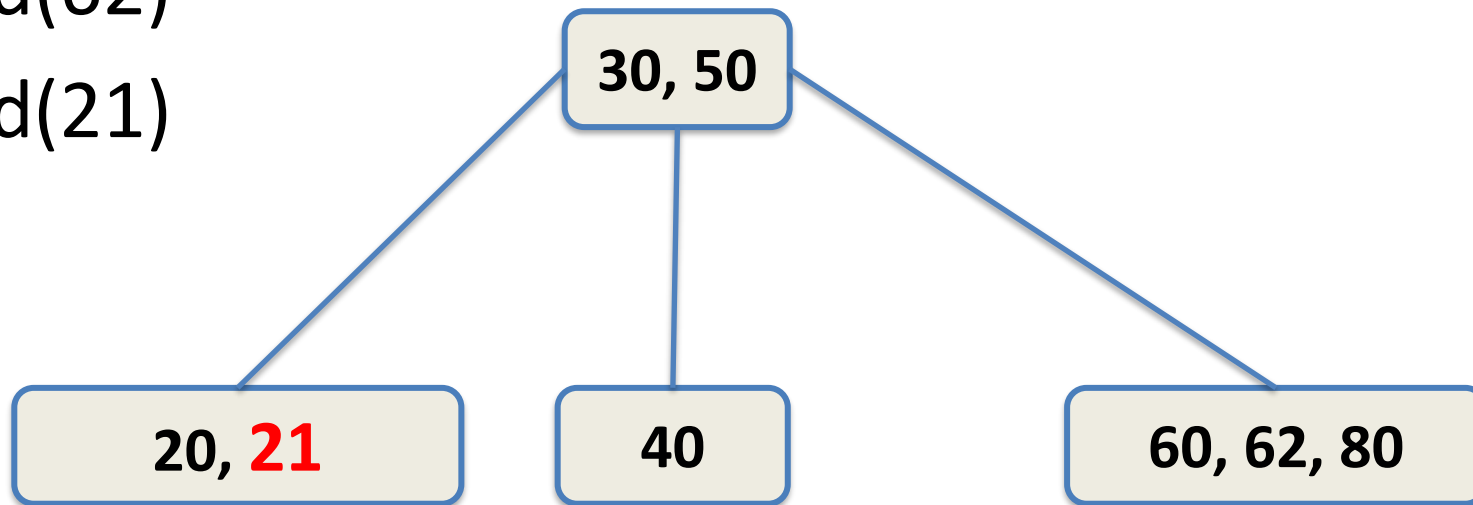
Overflow!

Eject 30 and **split** what's left

Adding to a 2-4 Trees

Add(62)

Add(21)



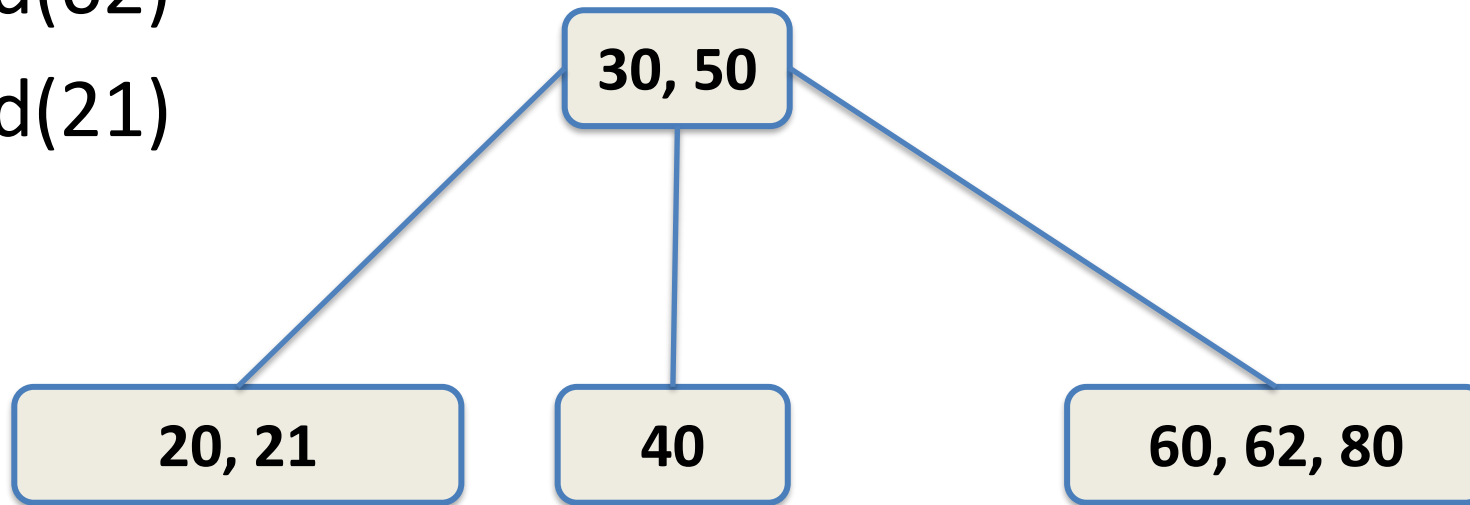
Overflow!

Eject 30 and **split** what's left

Adding to a 2-4 Trees

Add(62)

Add(21)

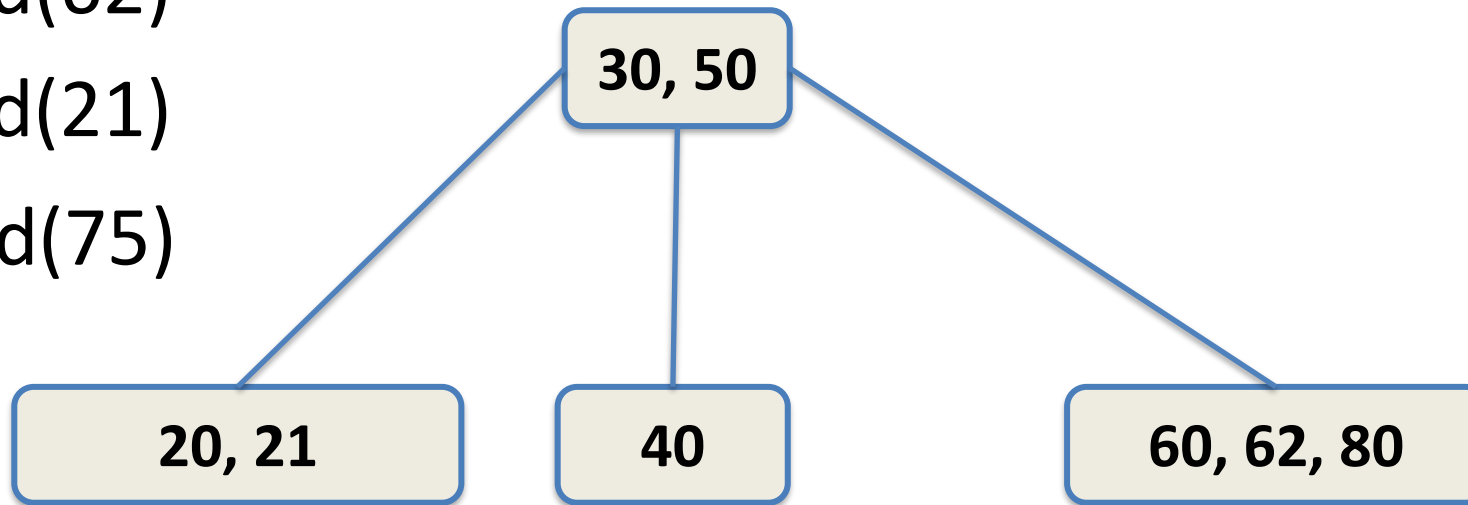


Adding to a 2-4 Trees

Add(62)

Add(21)

Add(75)

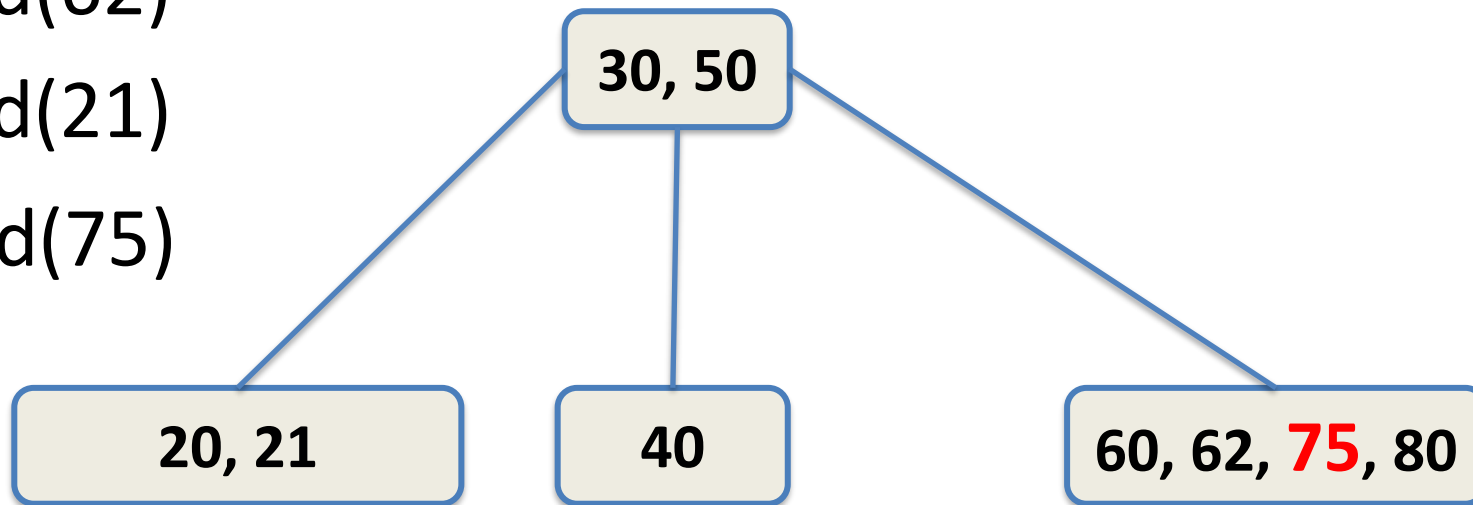


Adding to a 2-4 Trees

Add(62)

Add(21)

Add(75)



Overflow!

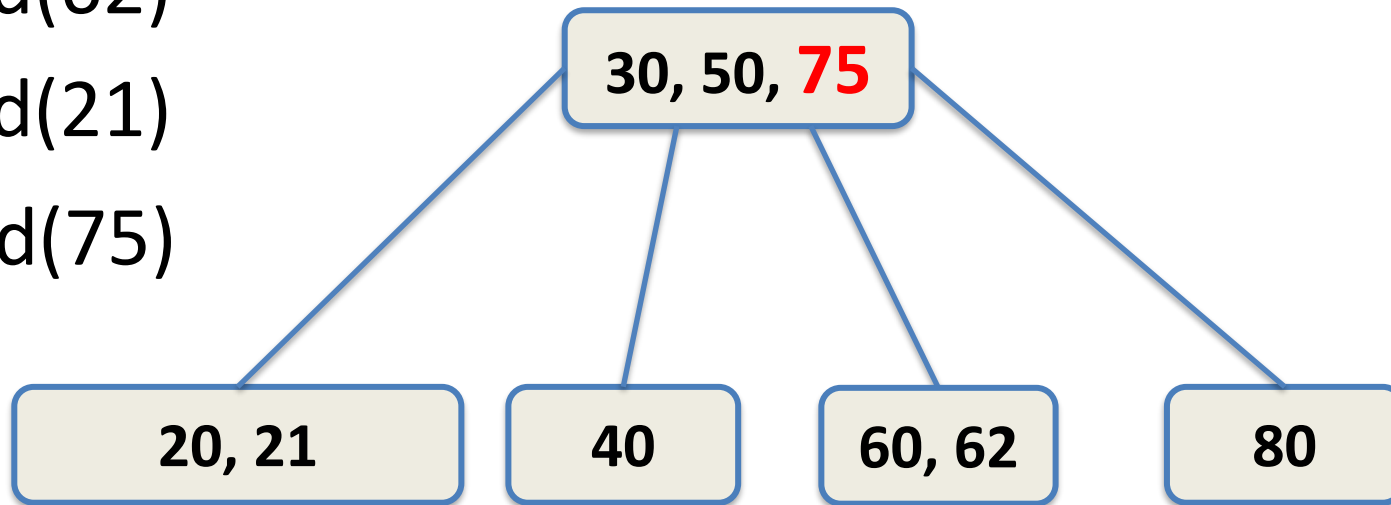
Eject 75 and **split** what's left

Adding to a 2-4 Trees

Add(62)

Add(21)

Add(75)



Overflow!

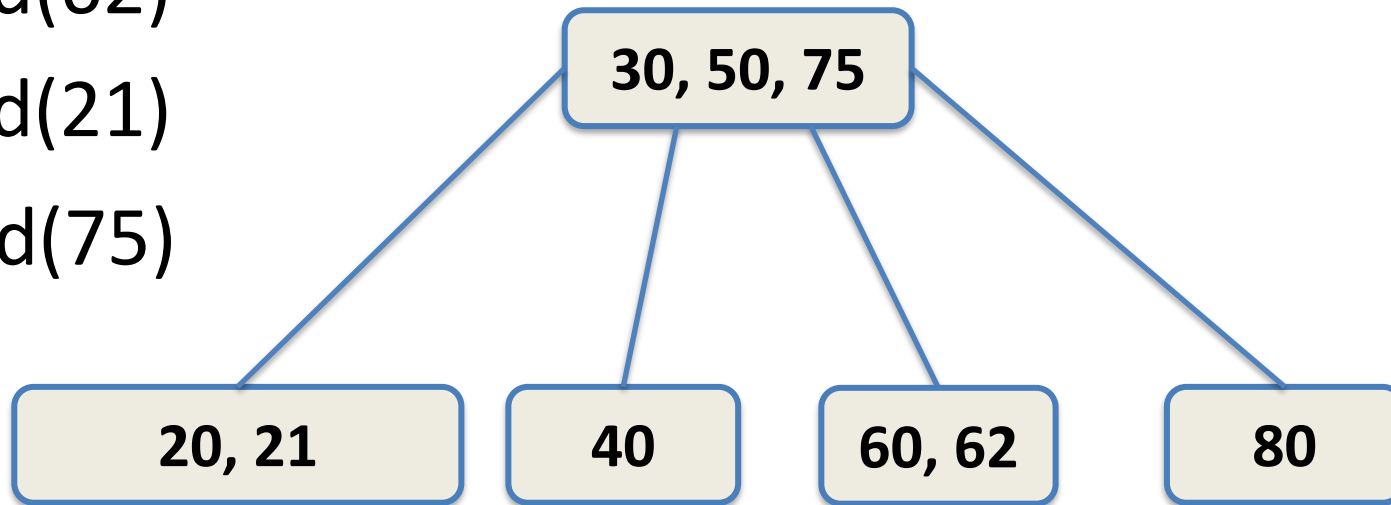
Eject 75 and **split** what's left

Adding to a 2-4 Trees

Add(62)

Add(21)

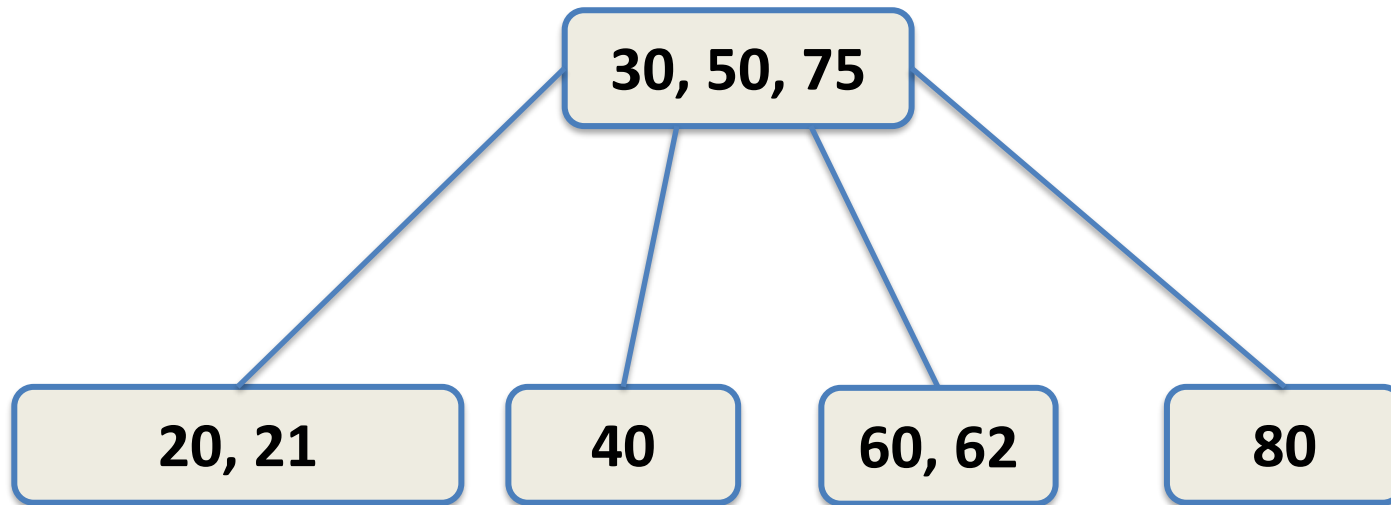
Add(75)



2-4 Trees

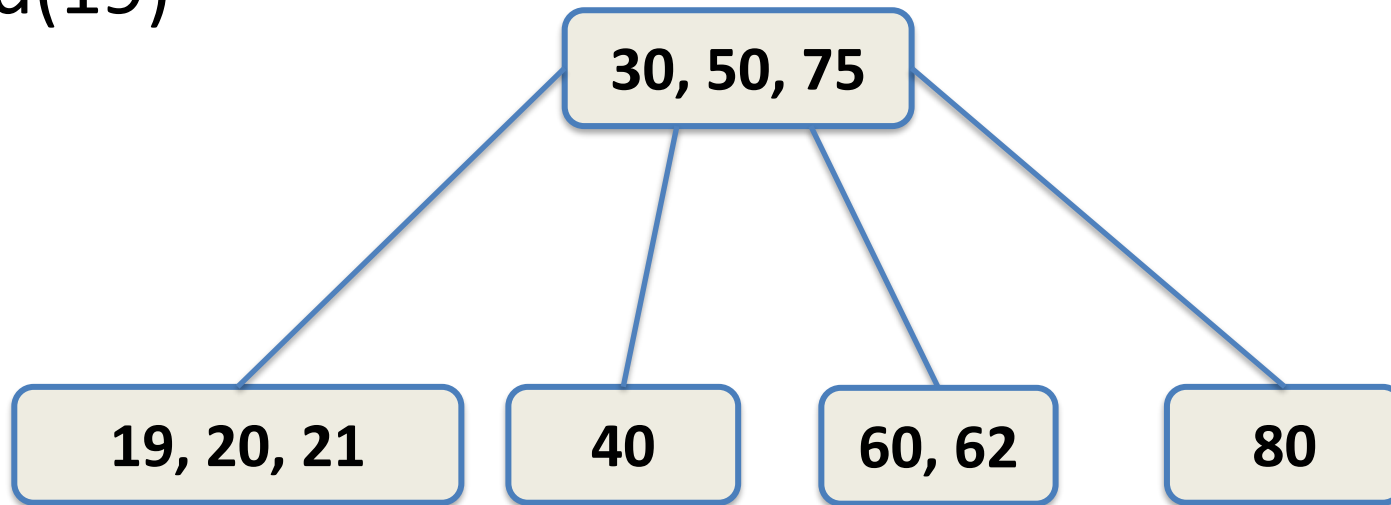
An **overflow** may cascade all the way up to the root node in the tree.

Adding to a 2-4 Trees



Adding to a 2-4 Trees

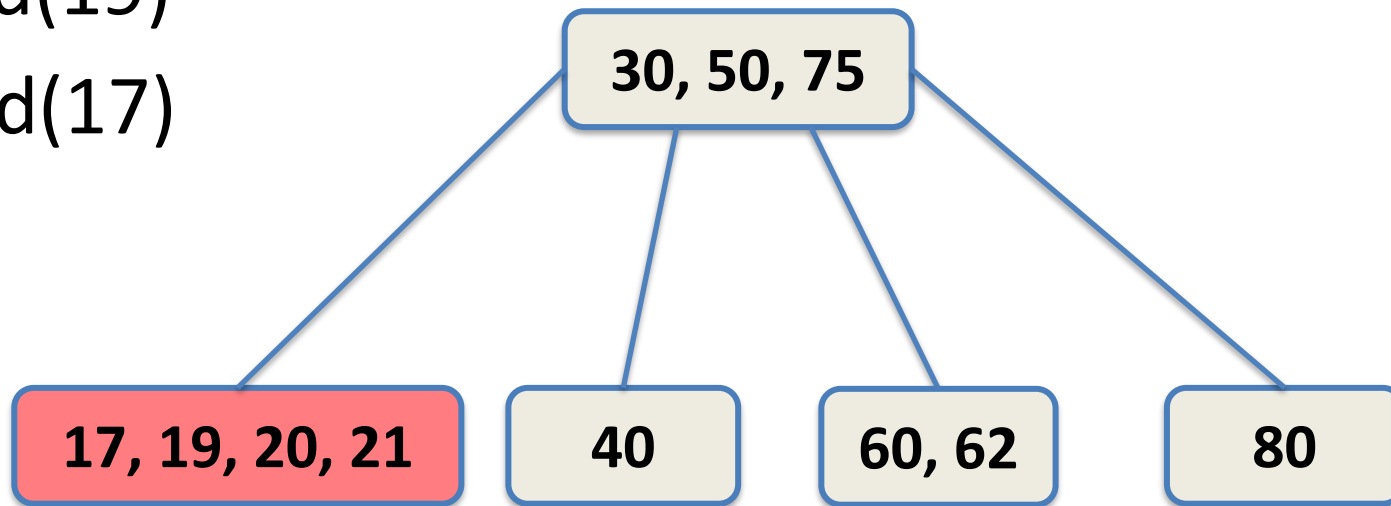
Add(19)



Adding to a 2-4 Trees

Add(19)

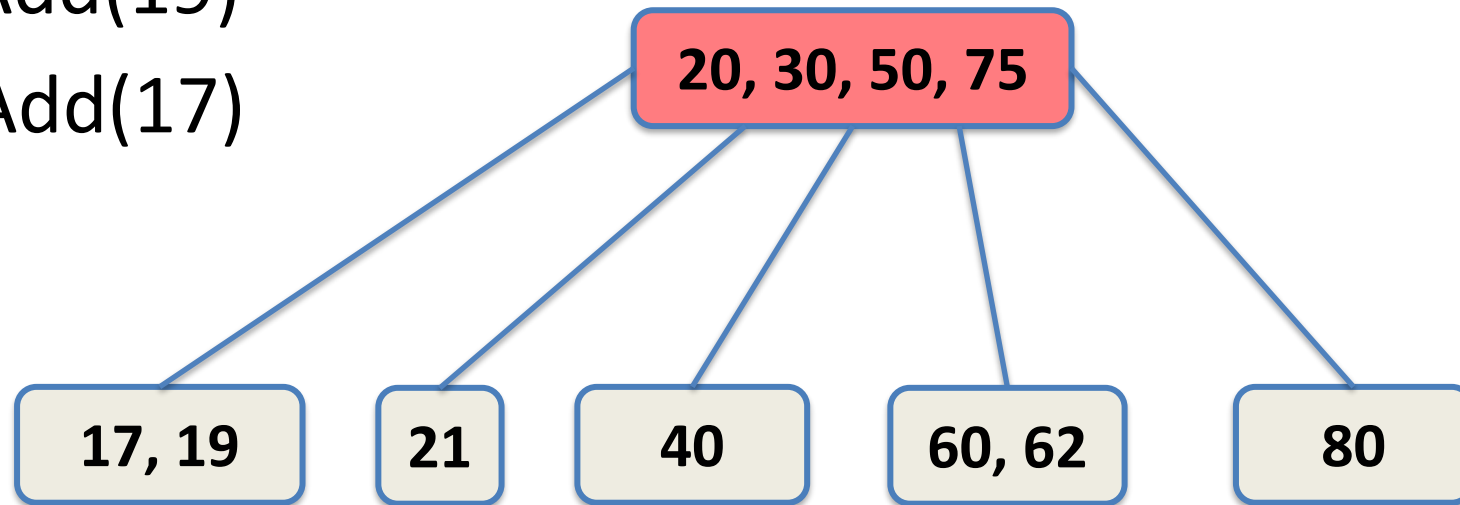
Add(17)



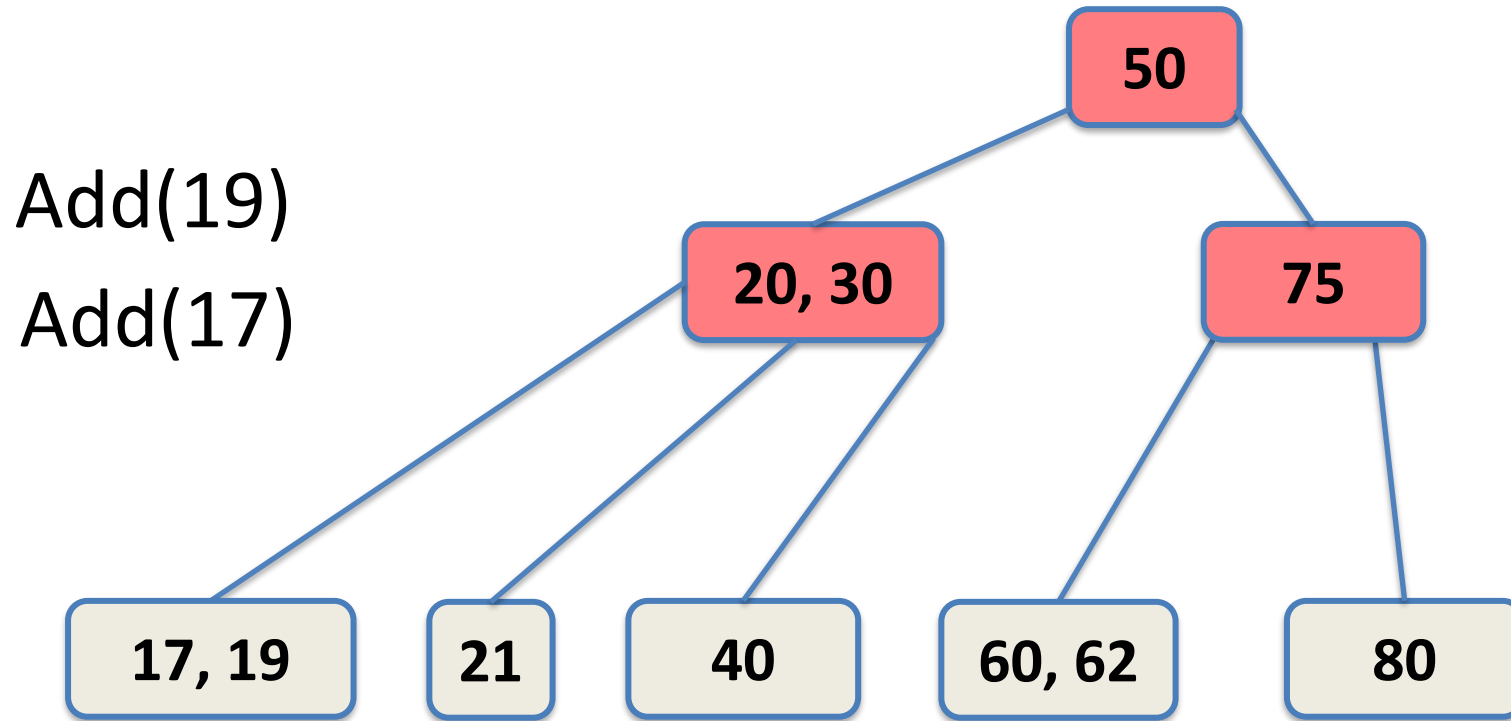
Adding to a 2-4 Trees

Add(19)

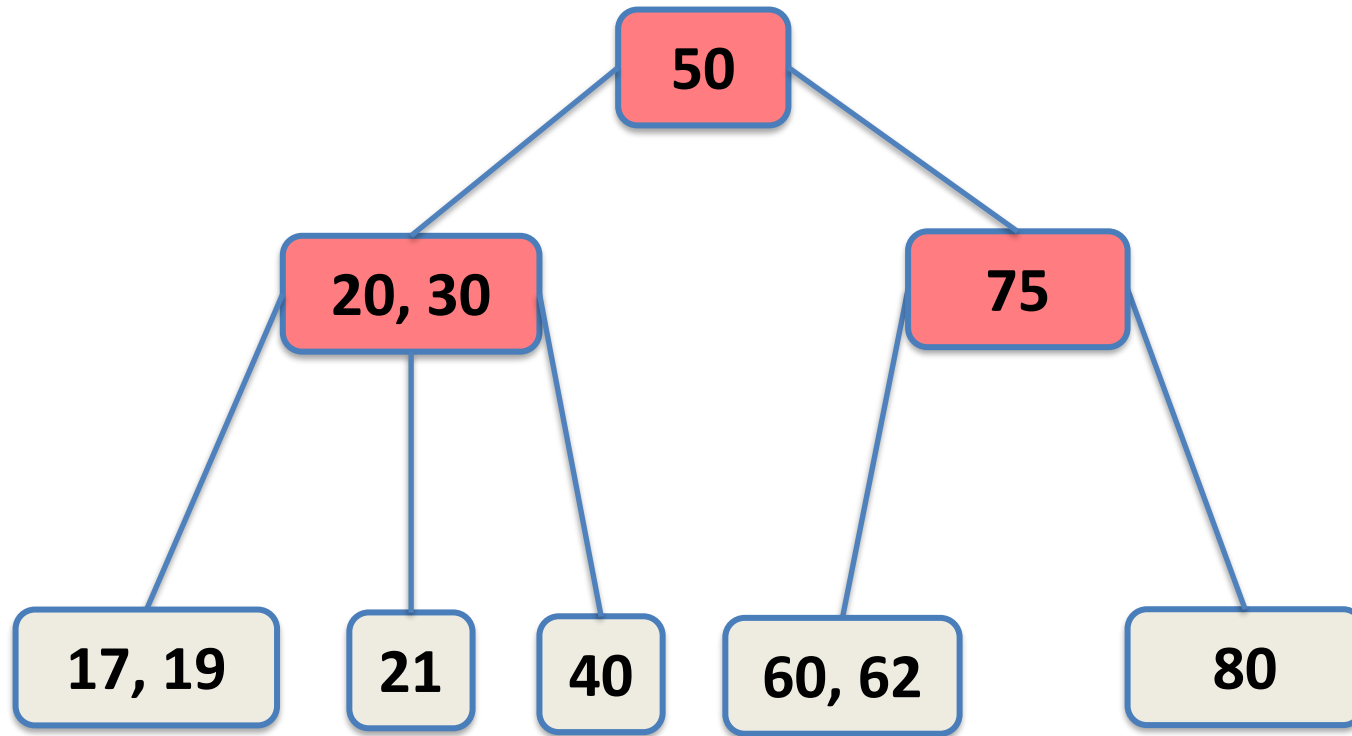
Add(17)



Adding to a 2-4 Trees



Adding to a 2-4 Trees



What is the cost of adding to a 2-4 tree?

2-4 Trees

Removing a value (key) to a 2-4 tree.

Just as with a BST you find the element to remove. If it is not at the bottom of the tree then you swap it with its in-order predecessor. (That is the biggest element that is smaller than it.)

You then remove the node from the bottom of the tree. An **underflow** occurs if that node would be empty after removing the key.

2-4 Trees

An **underflow** occurs when trying to remove a value from a node with only one key.

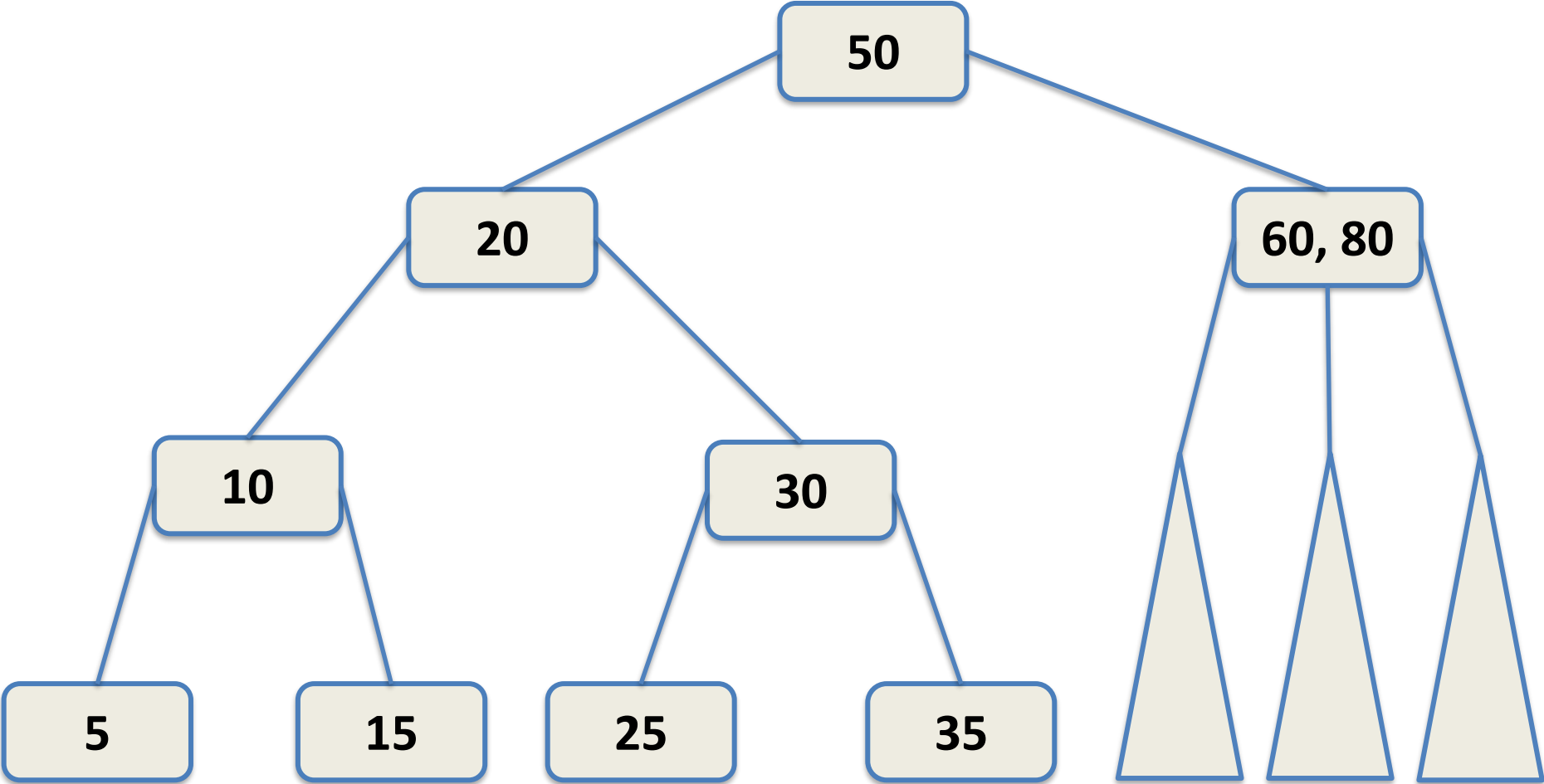
- If a sibling node has **enough** values we perform a **transfer**. We take a key from the parent and replace it with a key from the sibling node
- If sibling doesn't have enough values we perform a **fuse**. Merge (empty) node with sibling and take a key from the parent.

2-4 Trees

An **underflow** may cascade up the tree

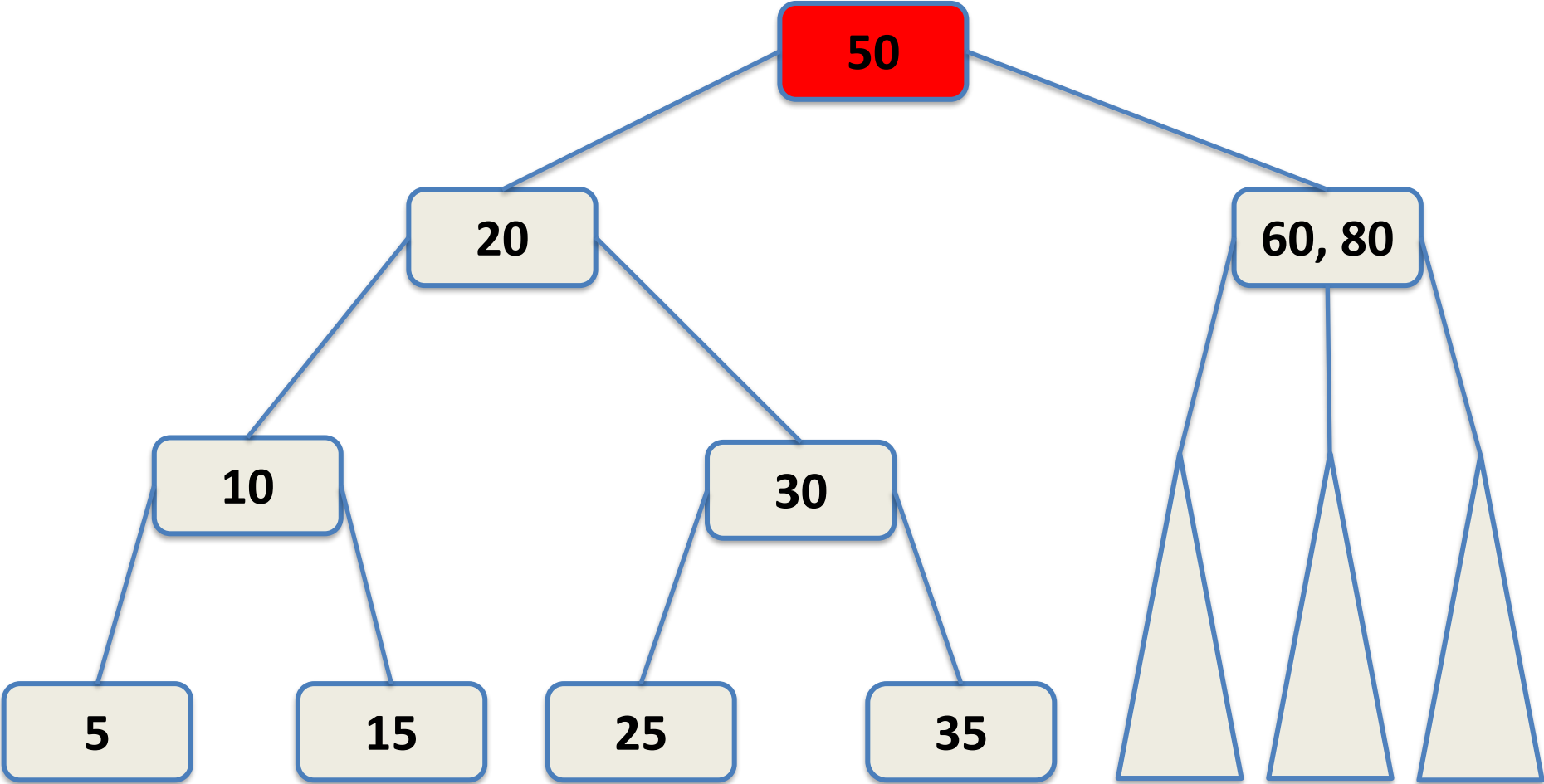
remove(50)

Removal in 2-4 Trees



remove(50)

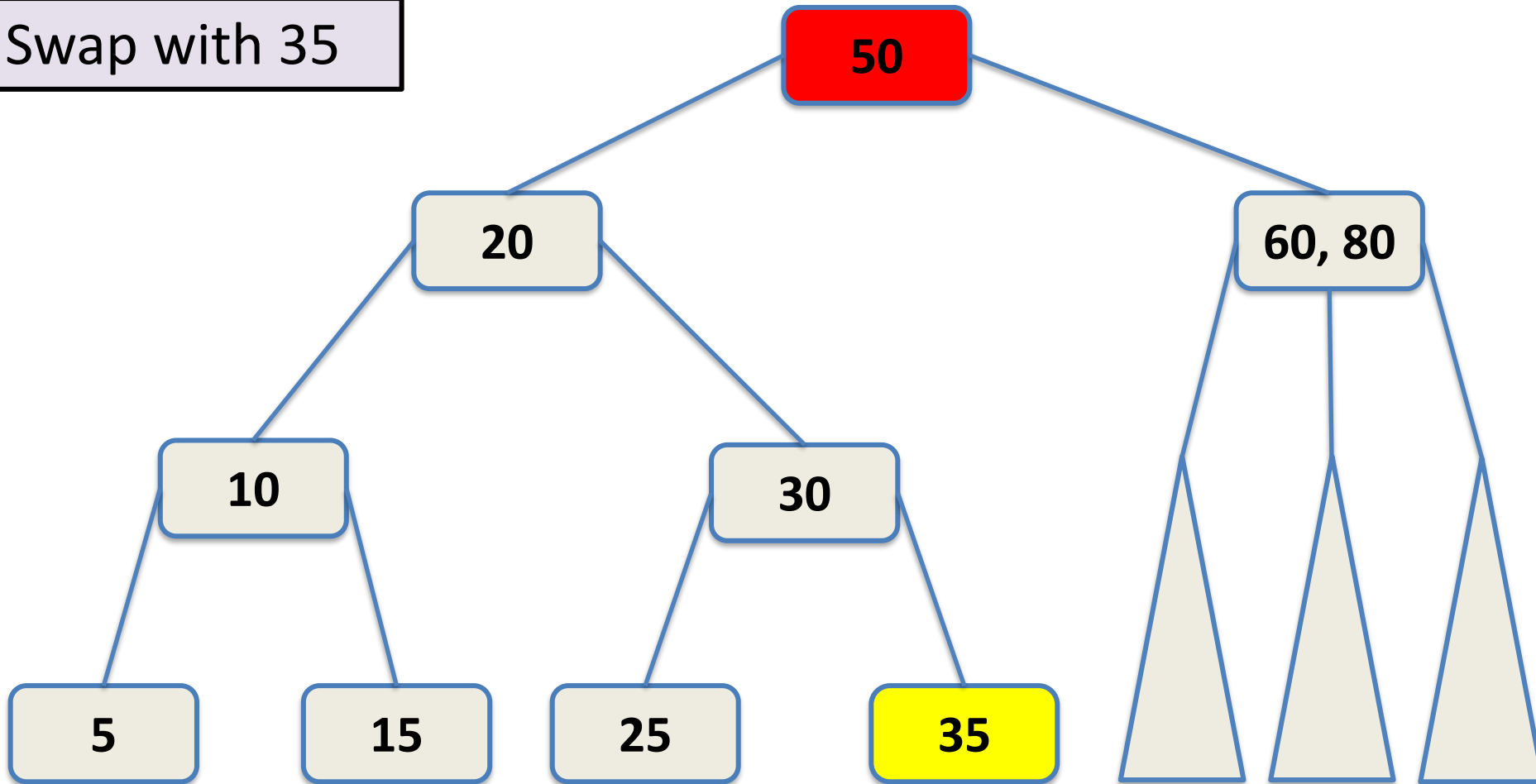
Removal in 2-4 Trees



remove(50)

Swap with 35

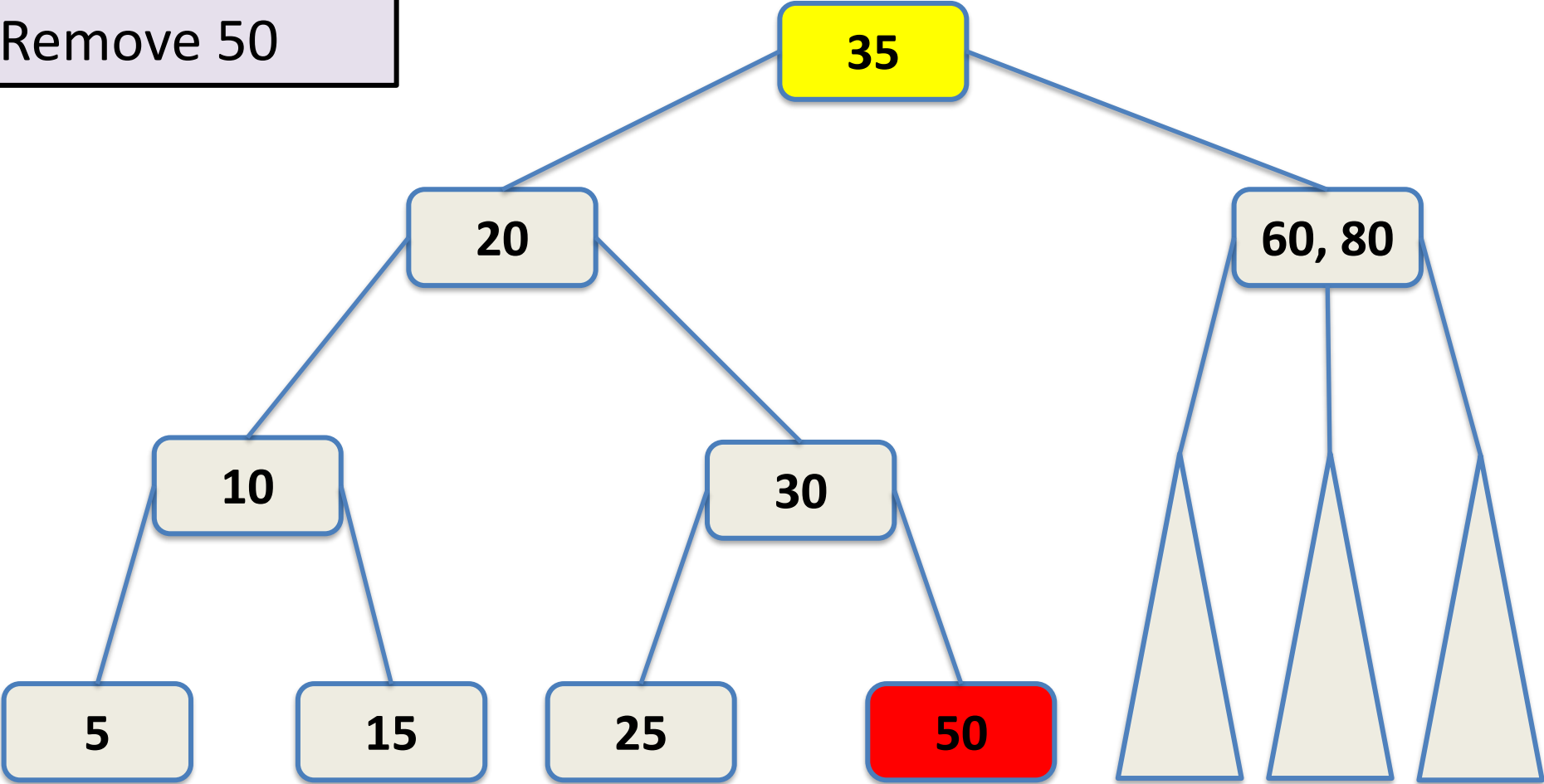
Removal in 2-4 Trees



remove(50)

Remove 50

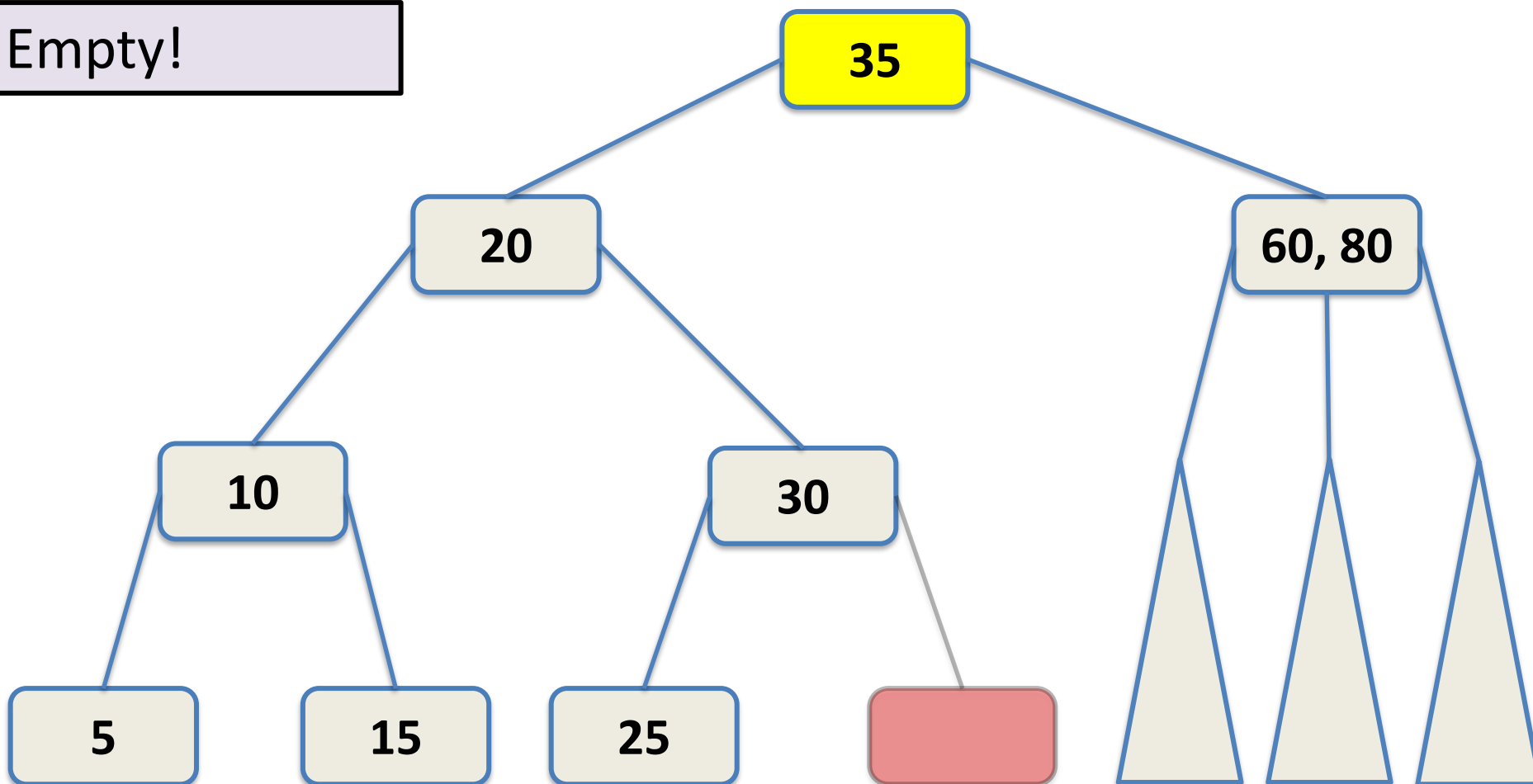
Removal in 2-4 Trees



remove(50)

Empty!

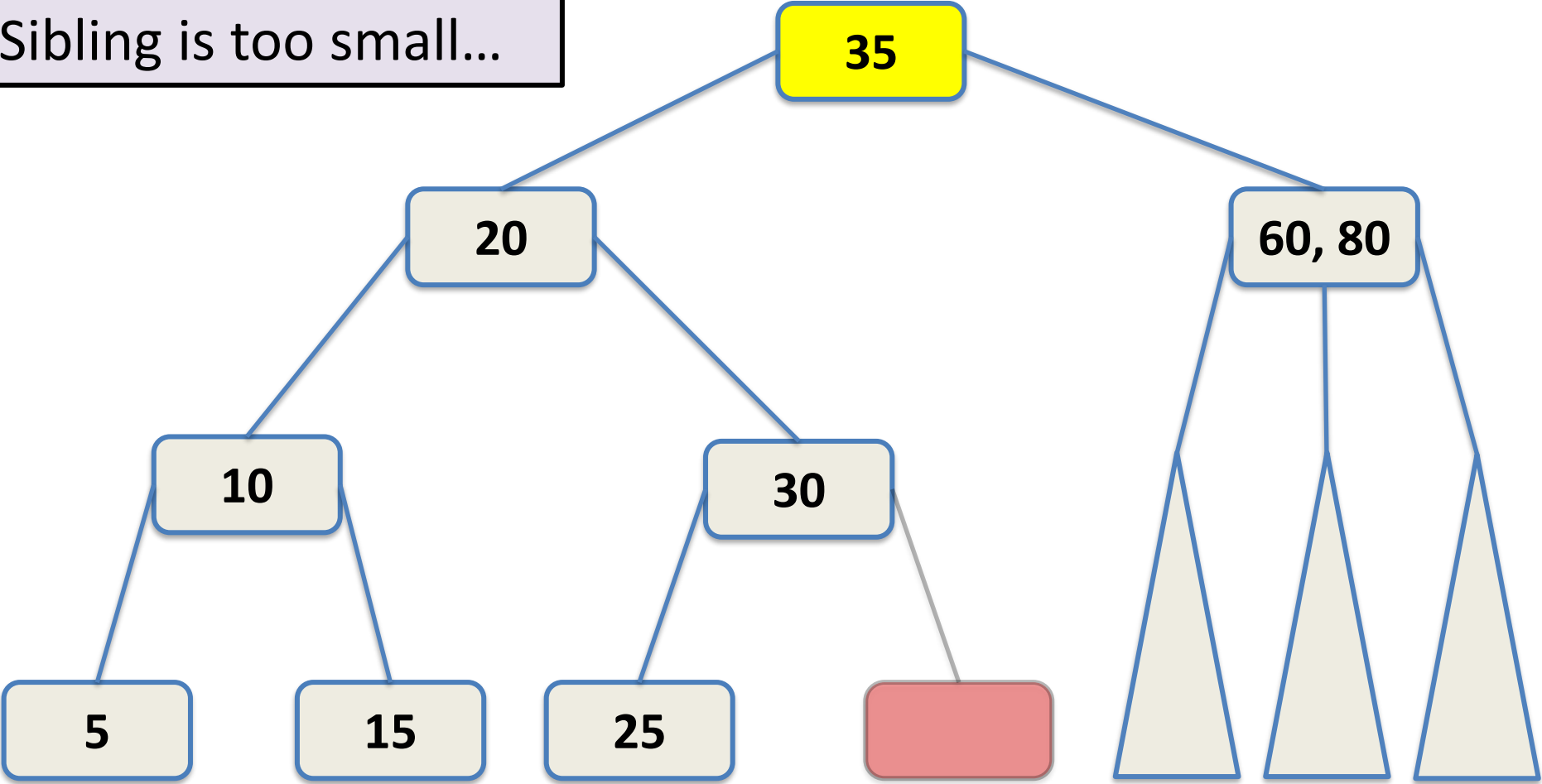
Removal in 2-4 Trees



remove(50)

Removal in 2-4 Trees

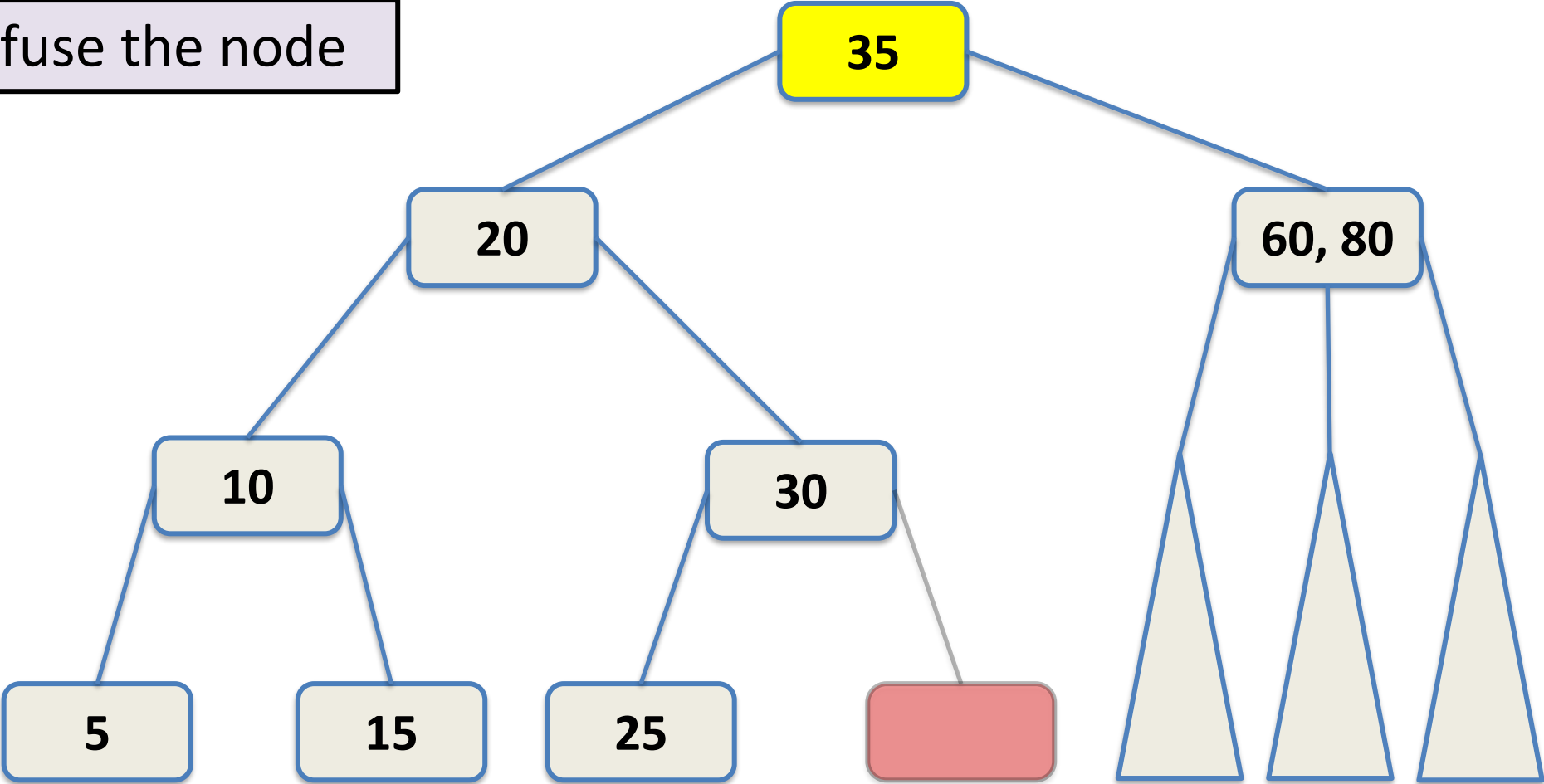
Sibling is too small...



remove(50)

fuse the node

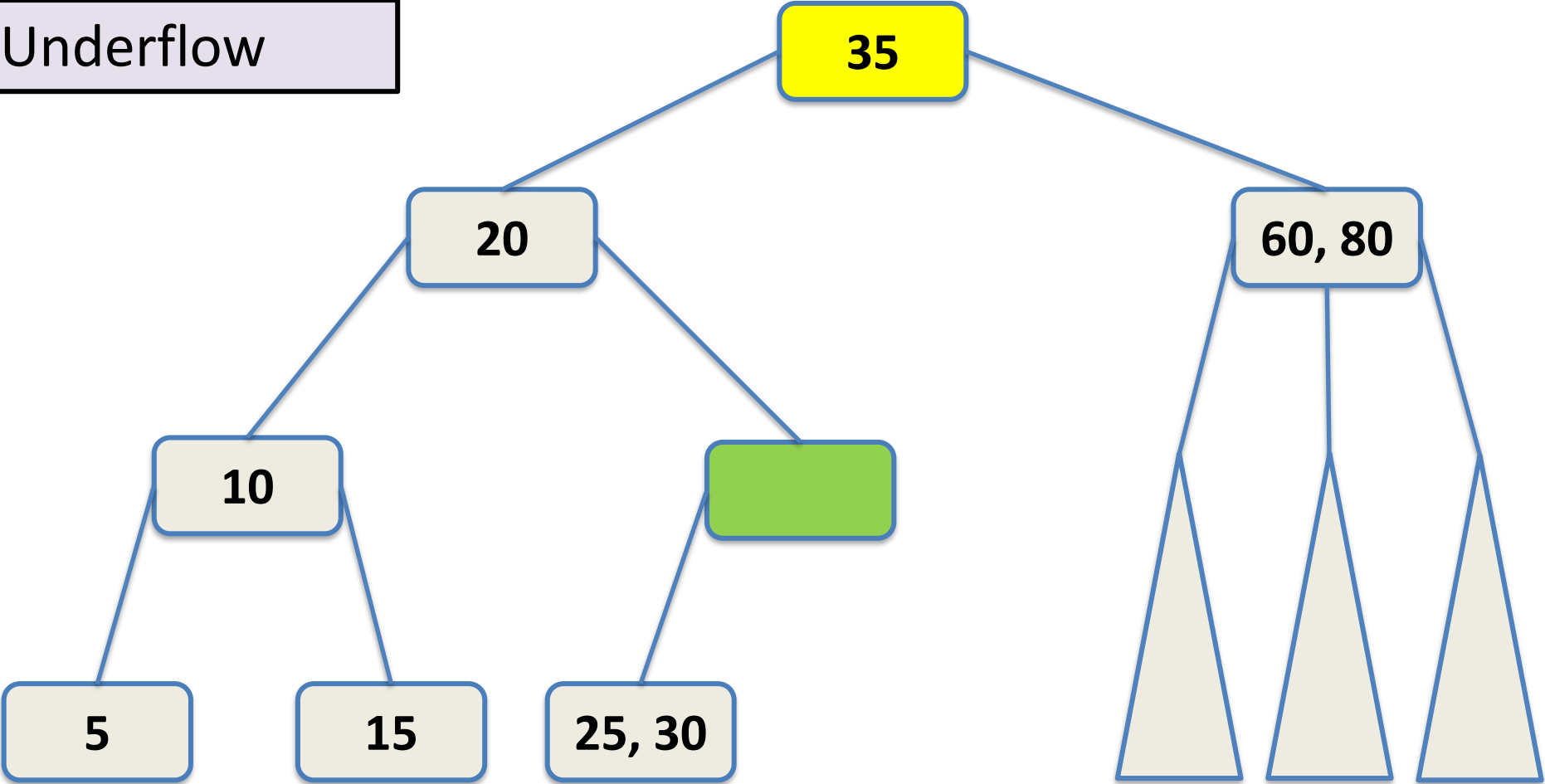
Removal in 2-4 Trees



remove(50)

Underflow

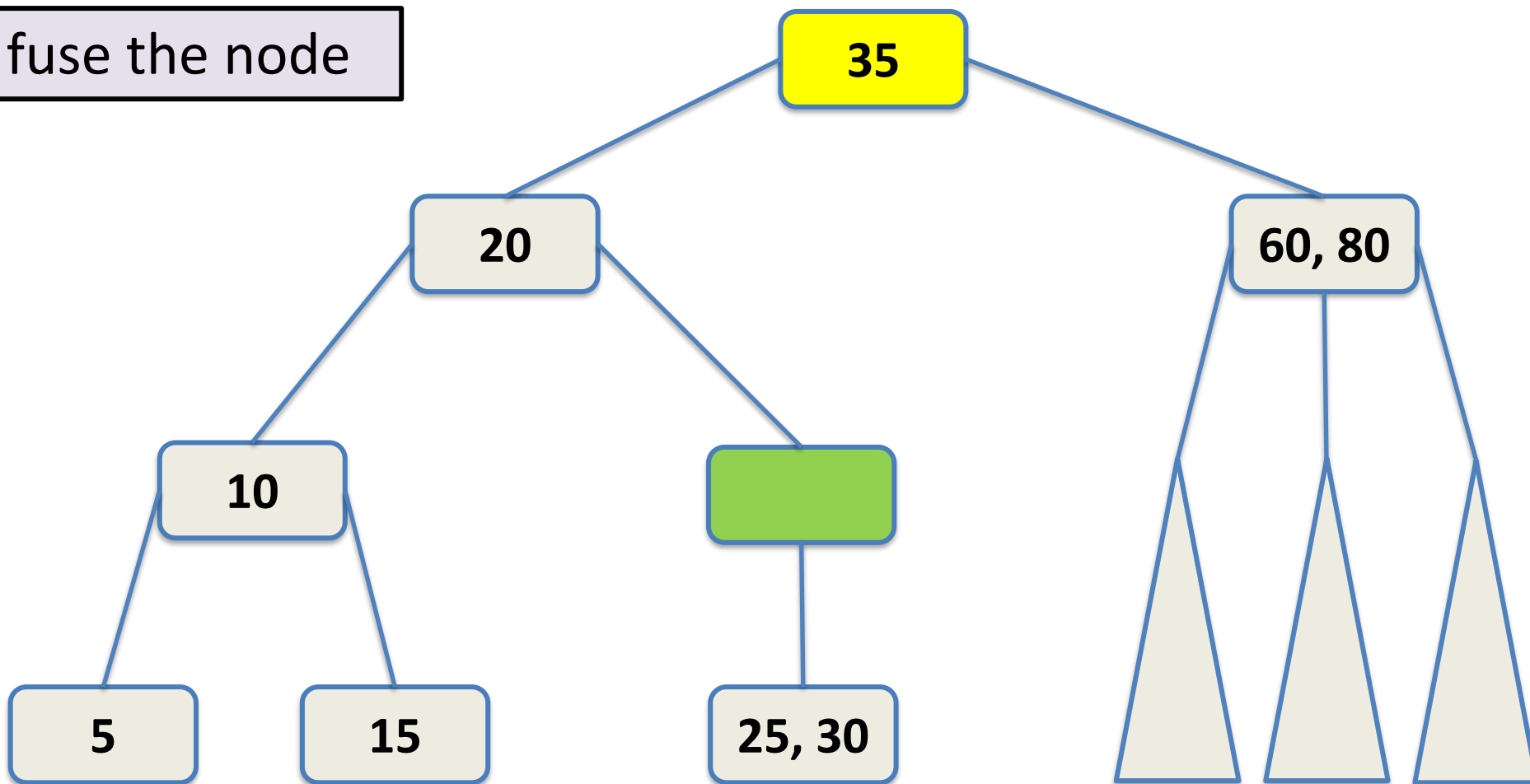
Removal in 2-4 Trees



remove(50)

fuse the node

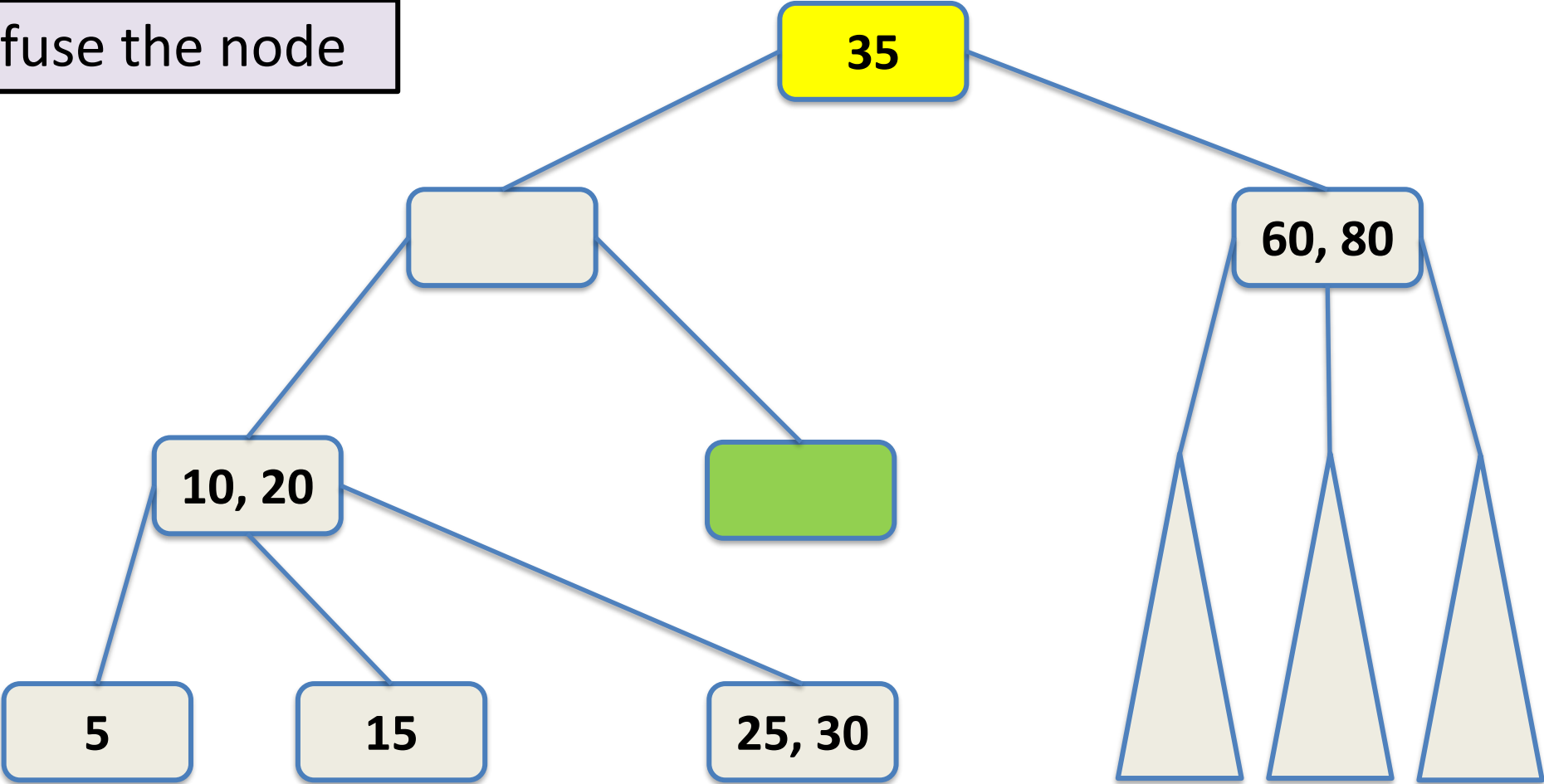
Removal in 2-4 Trees



remove(50)

fuse the node

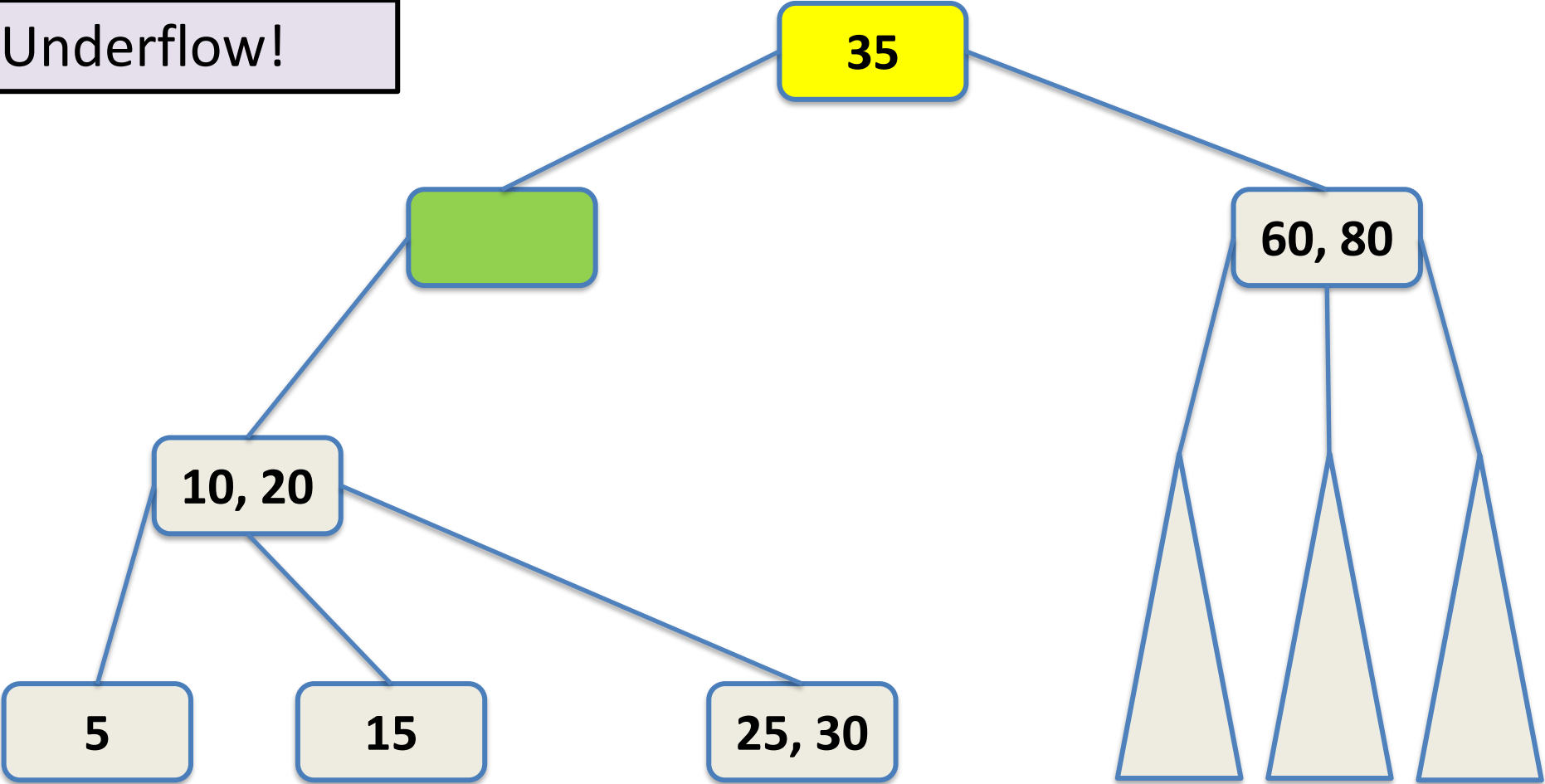
Removal in 2-4 Trees



remove(50)

Underflow!

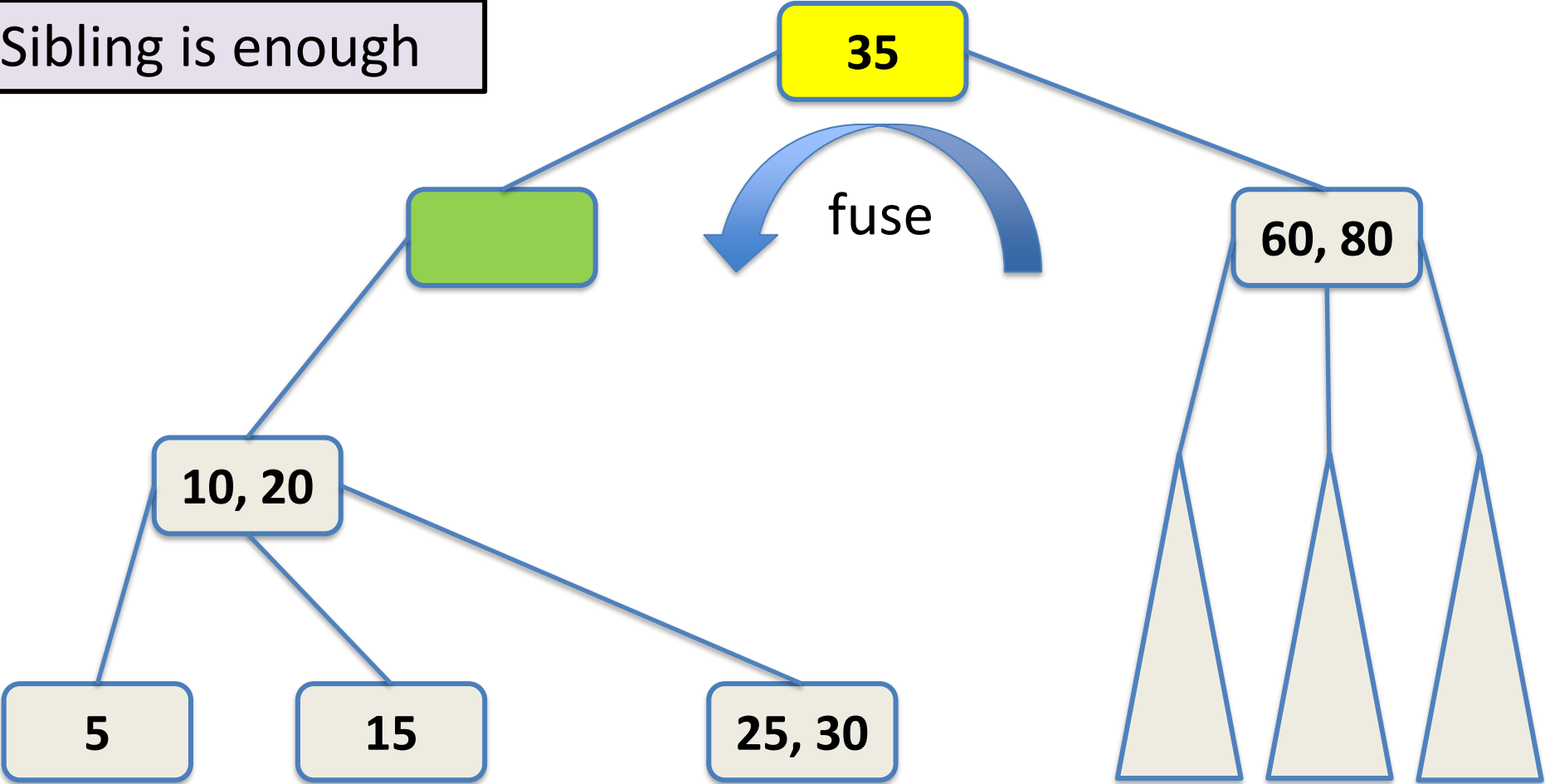
Removal in 2-4 Trees



remove(50)

Sibling is enough

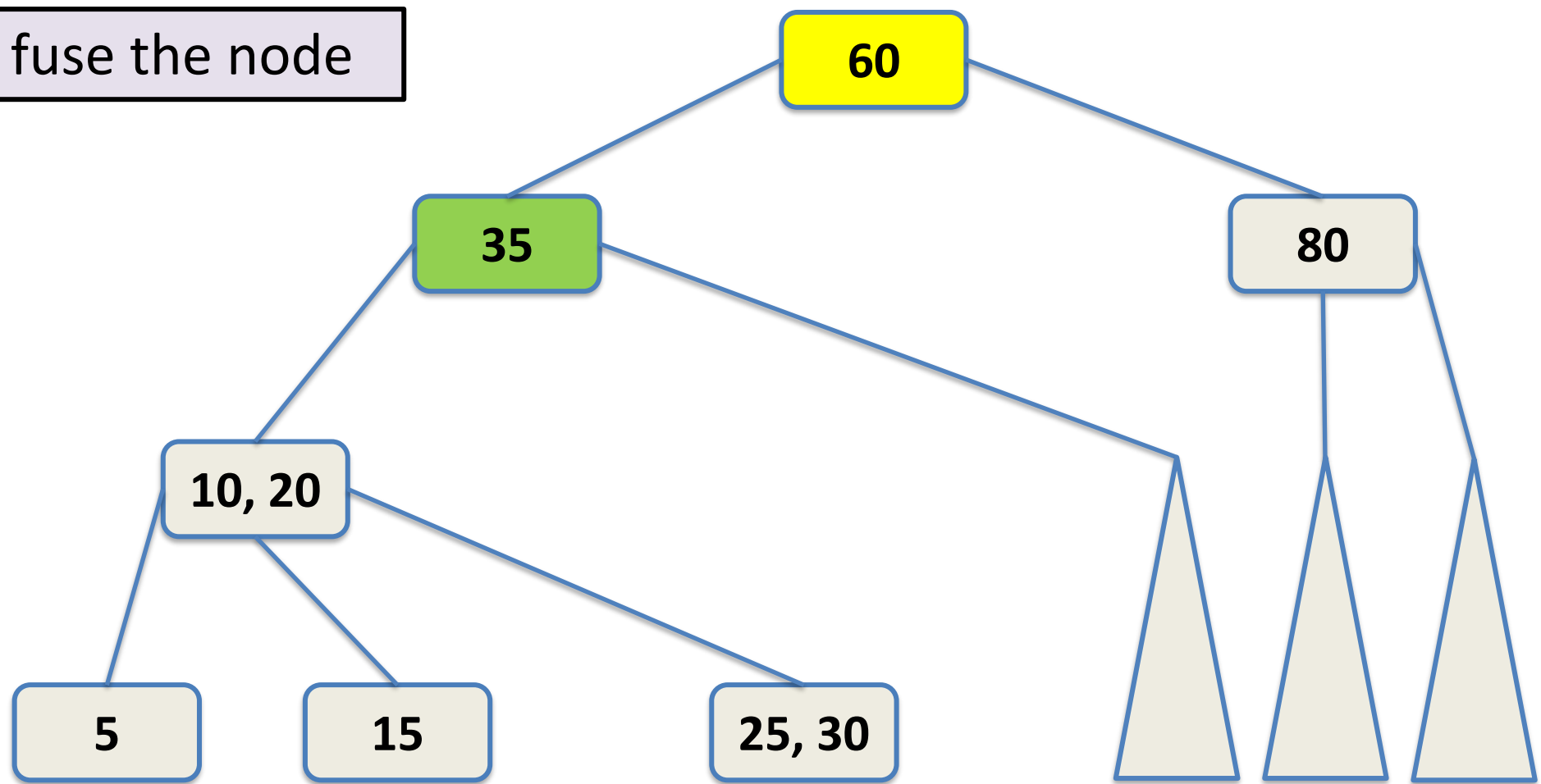
Removal in 2-4 Trees



remove(50)

fuse the node

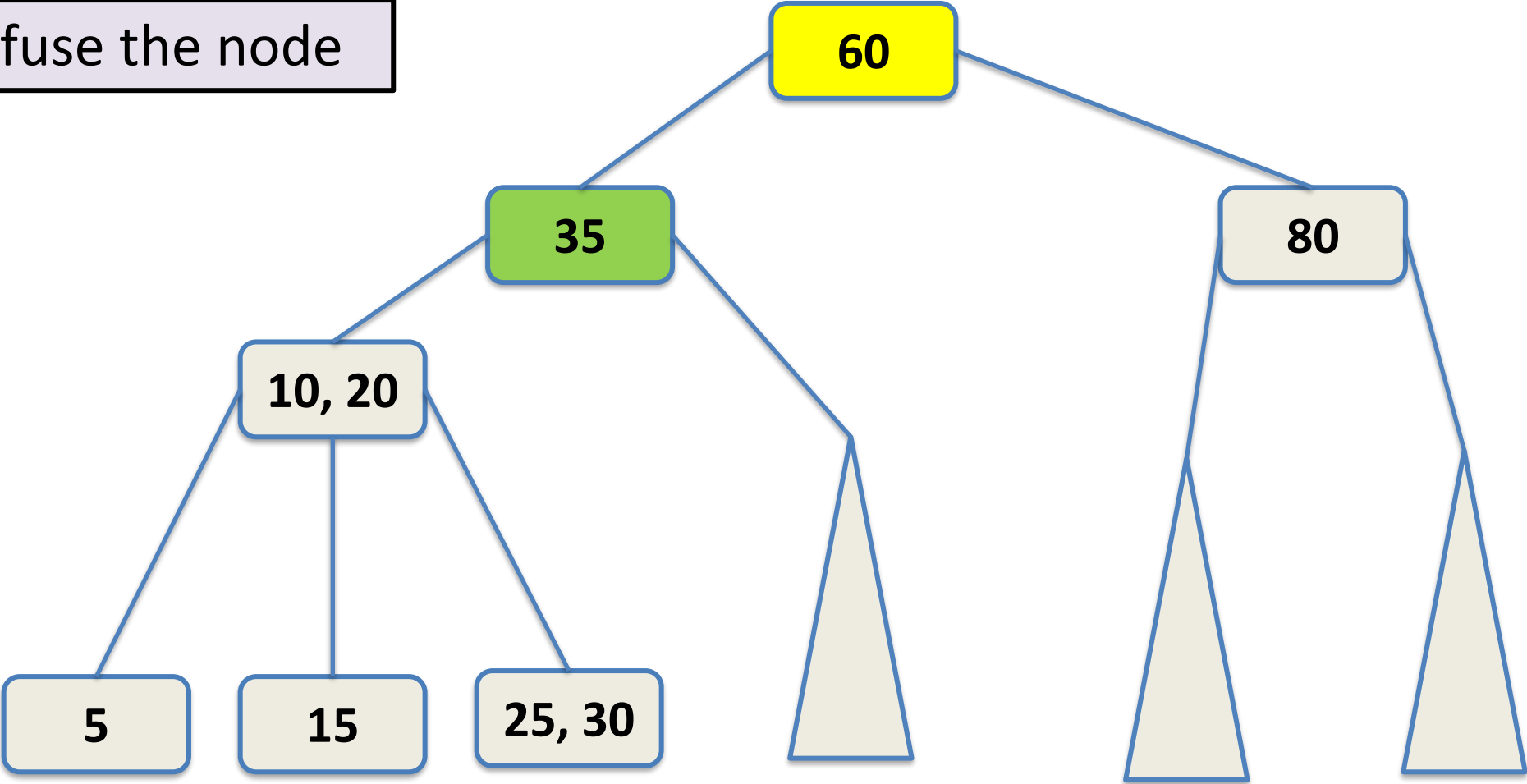
Removal in 2-4 Trees



remove(50)

fuse the node

Removal in 2-4 Trees



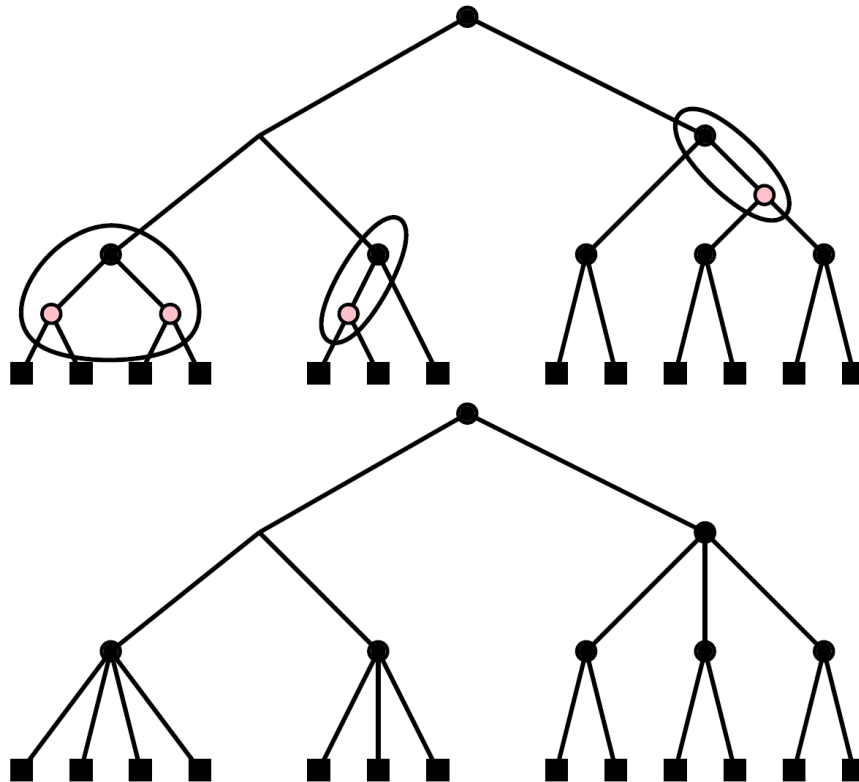
2-4 Trees

Lemma A 2-4 Tree implements the SSet interface and supports the operations `find()`, `add()`, and `remove()` in $O(\log n)$ time per operation.

Red Black Trees

A red black tree is a binary tree that simulates a 2-4 tree.

The nodes are coloured (labeled) either red (0) or black (1).



Red Black Trees

A binary search tree where nodes are coloured either red (0) or black (1) with the following properties:

Property 9.3 (black-height) There are the same number of black nodes on every root to leaf path. (The sum of the colours on any root to leaf path is the same.)

Property 9.4 (no-red-edge) No two red nodes are adjacent.

Property 9.5 (left-leaning) At any node u , if $u.\text{left}$ is black, then $u.\text{right}$ is black.

Red Black Trees

Theorem 9.1 A **RedBlackTree** implements the SSet interface and supports the operations `add(x)`, `remove()`, and `find(x)` in $O(\log n)$ worst-case time per operation.

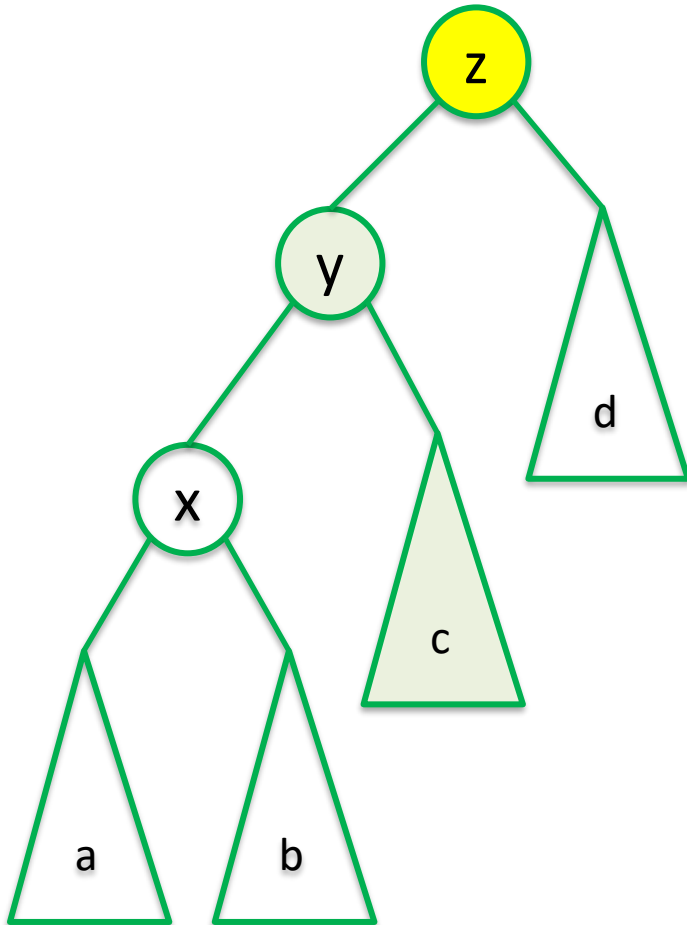
Theorem 9.2 Rebalancing has constant amortized cost.

AVL Trees

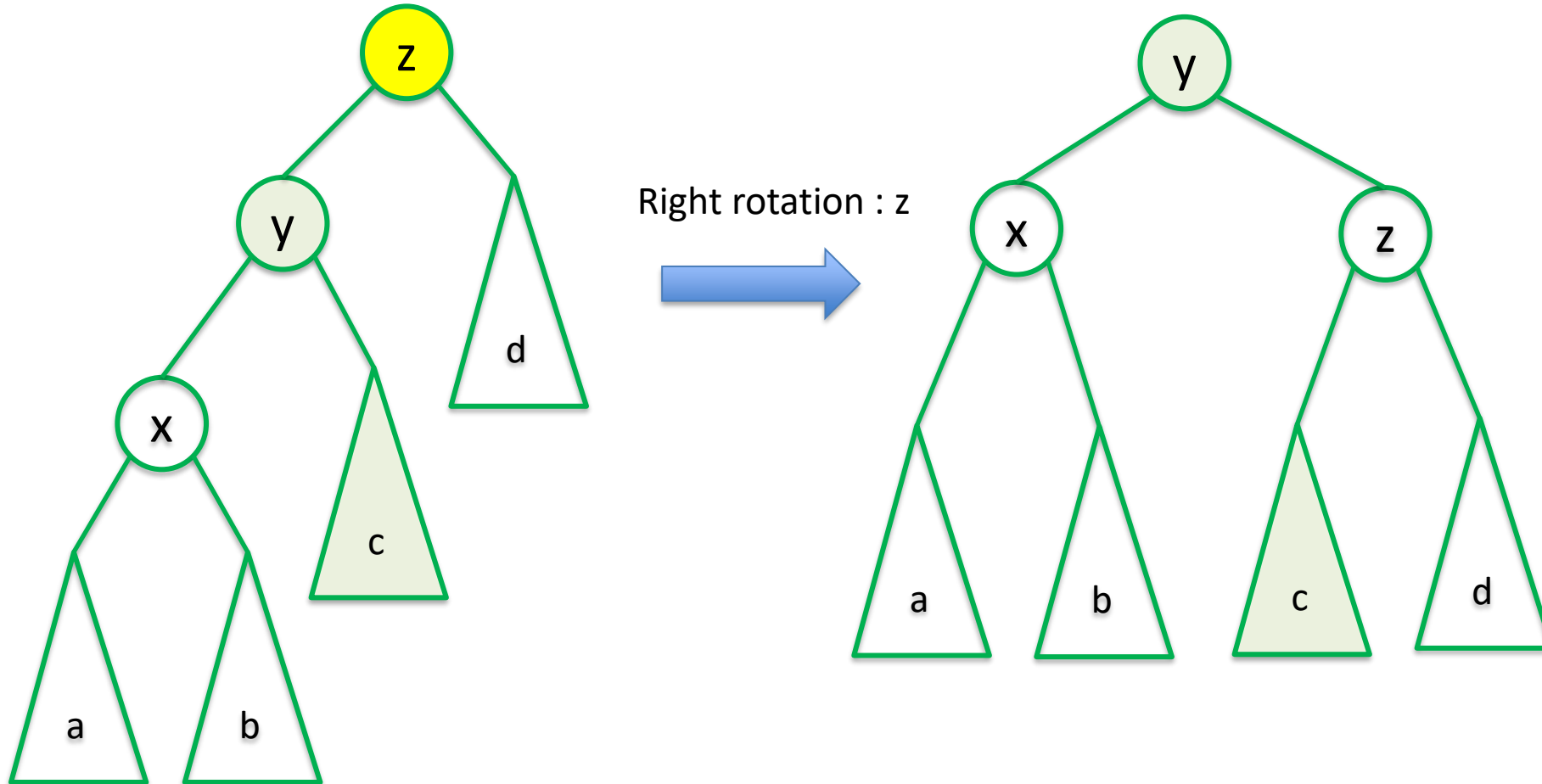
Another self-balancing binary search tree with the following properties:

1. Every node in the tree is almost balanced. The height (possibly zero) of the left and right sub-trees of any node differs by at most 1.
2. Every rooted sub-tree is an AVL tree

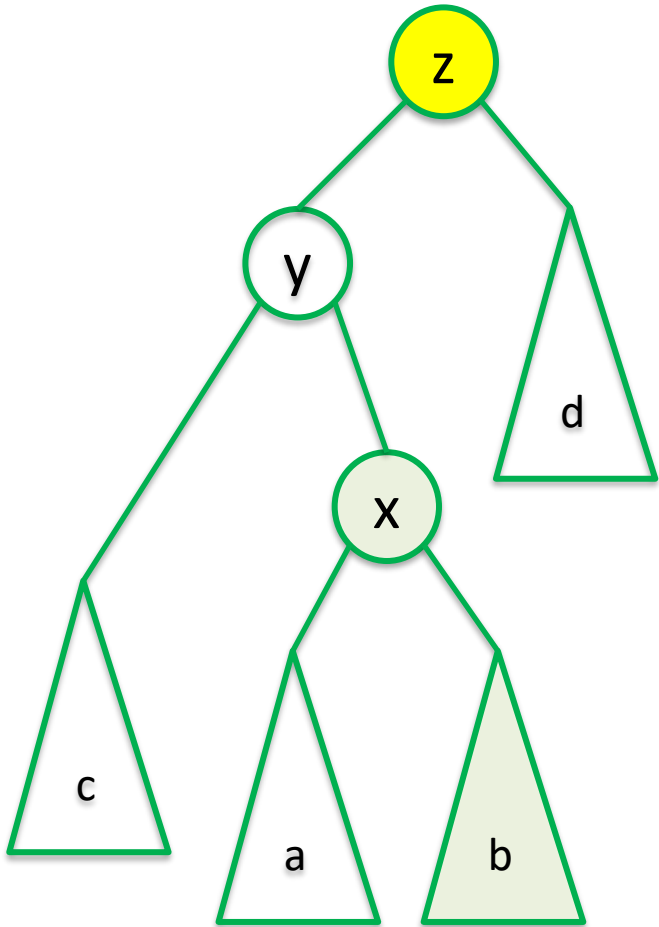
AVL Trees



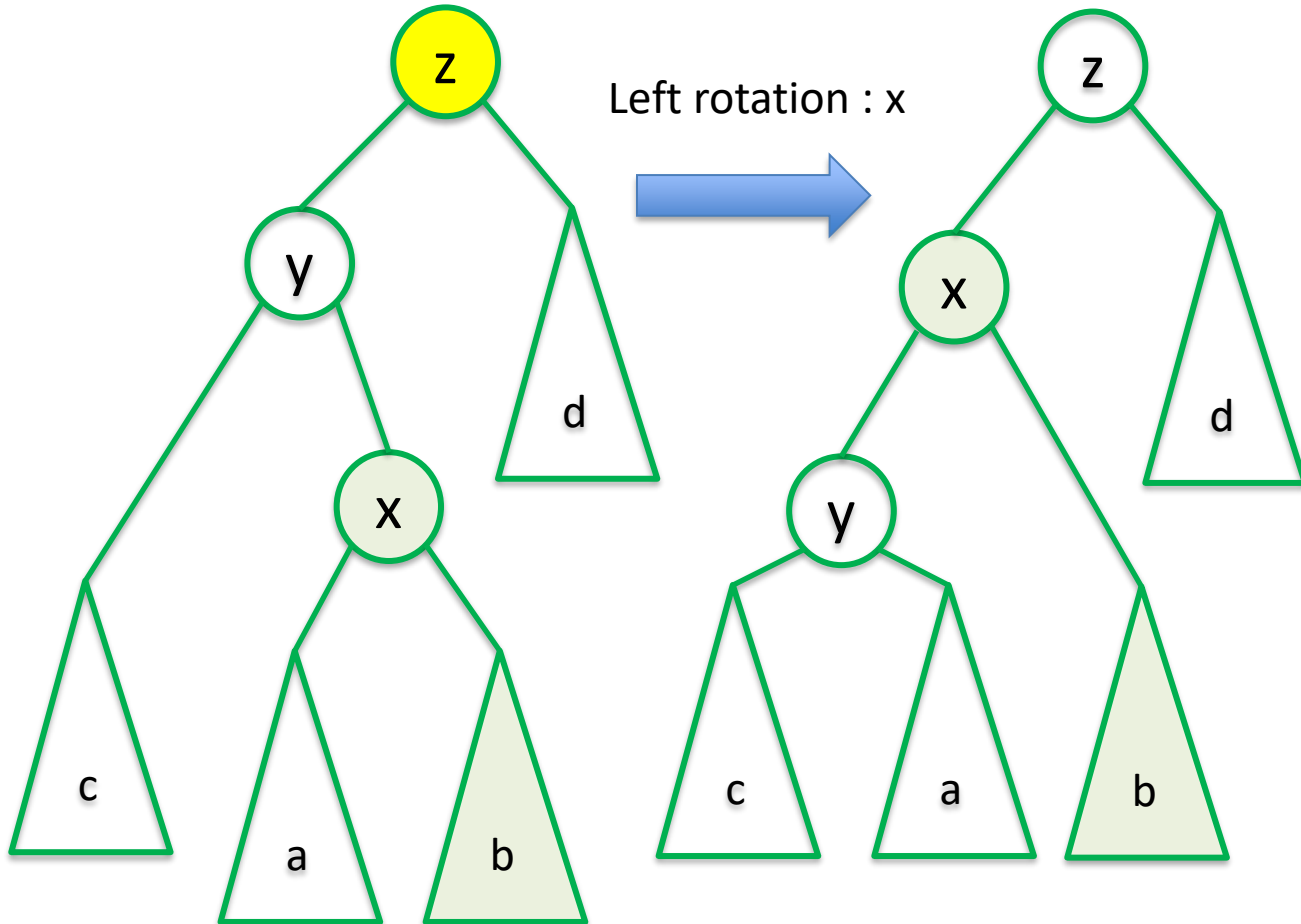
AVL Trees



AVL Trees



AVL Trees



Left Right

AVL Trees

